Phishing URL Detection on PhiUSIIL: A Leakage-Aware Comparison of Classical, Character-Level, and Clustering Methods

Pranav Dhinakar, Jason Justice, and and Guna Pasupathy

Shiley-Marcos School of Engineering

University of San Diego

AAI-501: Introduction to Artificial Intelligence

Professor [Instructor Name]

August 10, 2026

Author Note

Group 2 final team project. Source code and full commit history are available in the team GitHub repository: https://github.com/jjusticeusd/AAI-501-Final-Team-Project-Group2.git. A disclosure of generative-AI assistance appears in Appendix B.

Abstract

Phishing remains one of the most common vectors for scams and data breaches, and automated URL classification is a practical application of supervised learning. We build and evaluate a phishing-detection pipeline on the PhiUSIIL Phishing URL Dataset (235,795 labeled URLs, 54 features) and organize the work around a single finding: on this benchmark, near-perfect separability is intrinsic to the data rather than an artifact of any one leaky column or train/test split. We audit the dataset for leakage and confirm that content and lexical features reach roughly 99.9% accuracy with or without the obviously leaky `URLSimilarityIndex`. We then compare three classical classifiers (logistic regression, random forest, and XGBoost) with class-imbalance, tuning, feature-importance, and cost-sensitive analyses; a character-level convolutional network trained on the raw URL string alone as a leakage-immune "hard mode" comparison; and an unsupervised clustering of phishing rows into interpretable attack families. The character model reaches 0.9985 accuracy and 0.9993 PR-AUC using only URL characters, essentially tying the full-feature tabular models and demonstrating that the separability lives in the URL strings themselves. All supervised models land inside the published 99.5–100% PhiUSIIL band. We therefore frame the contribution not as a new accuracy ceiling but as an explanation of why the ceiling exists and what kinds of phishing the benchmark contains.

Phishing URL Detection on PhiUSIIL: A Leakage-Aware Comparison of Classical, Character-Level, and Clustering Methods

Introduction

Phishing is among the most common techniques used for scams and data breaches, and automated detection of malicious web addresses is a practical, high-impact application of supervised machine learning. This project builds and evaluates a system that classifies a URL as phishing or legitimate and, in doing so, compares more than one family of algorithms under a controlled experimental design, as the course project requires.

We use the PhiUSIIL Phishing URL Dataset (UCI Machine Learning Repository, id 967), which contains 235,795 labeled URLs described by 54 features. The target column `label` encodes 1 for legitimate and 0 for phishing. The dataset is large, has no missing values, and ships with pre-engineered lexical and page-content features, so it satisfies the project's size and preprocessing constraints without heavy cleaning.

The intellectual core of the project is a leakage-first framing. PhiUSIIL is known to be "too easy" when every column is handed to a model, and a naive report could stop at a near-perfect accuracy number. We instead treat that near-perfection as the thing to be explained. Our central finding, established three separate ways across the notebooks, is that the separability of phishing from legitimate URLs is a property of the feature set and even of the raw URL strings, not a side effect of one leaky column or a favorable split. Every modeling result in this report is read through that lens.

The work maps onto several topics from the course. It uses supervised binary classification and ensemble methods (logistic regression, random forest, and gradient boosting), feature selection and model-based feature importance, and model evaluation under mild class imbalance (precision, recall, F1, ROC-AUC, and PR-AUC). It connects to deep learning through a character-level convolutional network applied to raw URL text, which also touches the NLP idea of learning from character sequences. Finally it uses unsupervised clustering (KMeans and HDBSCAN) to characterize structure within the

phishing class, satisfying the requirement to combine at least two different algorithm types.

The report follows the three notebooks that make up the project code. Section 2 presents the data audit and exploratory analysis (Notebook 01, Guna Pasupathy), which establishes the leakage-clean feature set and split conventions used everywhere else. Section 3 presents the classical models, class-imbalance handling, tuning, feature importance, and a cost-sensitive decision layer (Notebook 02, Jason Justice). Section 4 presents the grouping decision, the character-level model, the clustering of attack families, and the state-of-the-art comparison (Notebook 03, Pranav Dhinakar). Section 5 synthesizes the three notebooks against the leakage thesis and the project requirements, and Section 6 concludes.

Data, Leakage Audit, and Exploratory Analysis

This section reproduces the analysis and commentary of Notebook 01 (author: Guna Pasupathy). The dataset is the PhiUSIIL Phishing URL Dataset (UCI id 967): 235,795 URLs, 54 features, with `label` *equal to 1 for legitimate and 0 for phishing. Because this dataset is known to be "too easy" if every column is thrown at a model, the notebook checks for data leakage before any modeling: it loads the data, looks at class balance and duplicates, checks the leaky and suspicious columns (*`URLSimilarityIndex`*, too-perfect yes/no features, repeated domains,* `TLD`*), and saves the list of safe feature columns for Notebooks 02 and 03.*

There are no missing values at all, which matches the UCI description. The four string columns are `URL`, `Domain`, `TLD`, and `Title`.

Class Balance

About 57% of the rows are legitimate and 43% are phishing (Figure 1). The proposal feedback assumed a heavily imbalanced dataset, but this is only a mild imbalance, so aggressive resampling is not needed. The plan is to use class weights in the models and to try SMOTE only as a comparison experiment (Notebook 02).

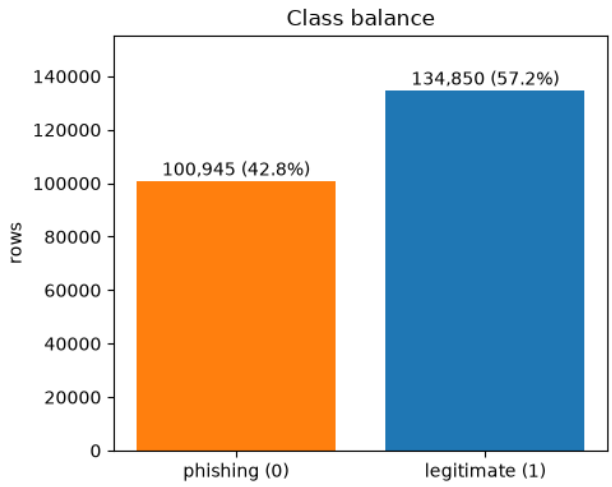

Figure 1

Class balance in PhiUSIIL. Legitimate URLs (label 1) slightly outnumber phishing URLs (label 0), a roughly 57/43 split.

Duplicate Rows and Duplicate URLs

There are no fully duplicated rows, but 425 URLs appear more than once (always with the same label). If the same URL lands in both train and test, the model gets tested on something it literally saw during training. Notebook 02 therefore drops duplicate URLs before splitting.

Leakage Check 1: `URLSimilarityIndex`

This column measures how similar a URL is to known legitimate URLs, so it basically contains the answer. Every single legitimate URL scores exactly 100 (standard deviation 0), which is clearly leakage. It is not perfect in the other direction though: 786 phishing URLs also score 100, so the column alone cannot reach 100% accuracy (Figure 2). The log scale is needed because the legitimate spike at 100 (134,850 rows) would otherwise flatten the phishing distribution.

How much does this one column matter? A quick logistic regression shows the leaky column alone gets 99.6%. But here is the surprise: dropping it barely matters, because the remaining roughly 50 features together get 99.9%. So the dataset is almost trivially easy even without the leaky column. That makes sense (features like `HasPasswordField` or

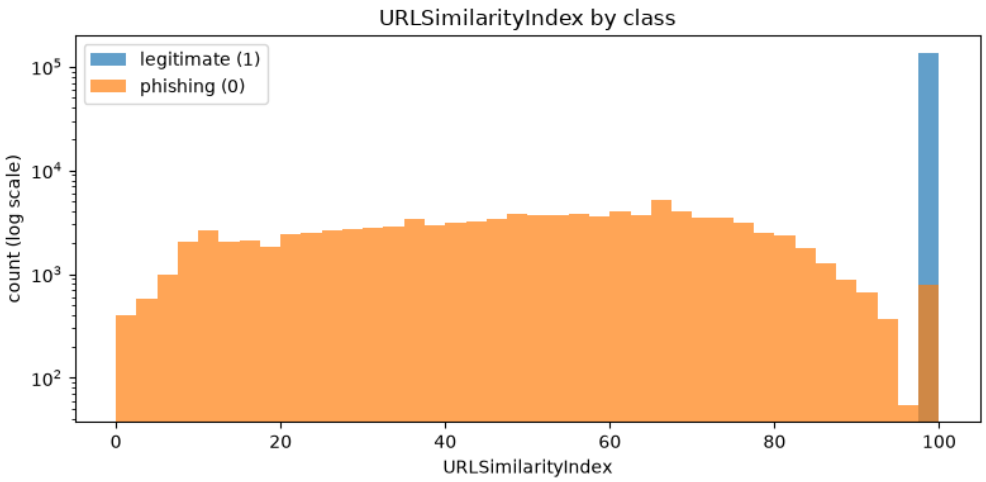

Figure 2

Distribution of ***`URLSimilarityIndex`*** *by class (log-scaled counts). Legitimate URLs pile up at exactly 100; a minority of phishing URLs also reach 100.*

`LineOfCode` come from crawling the actual page), but it means the tabular models solve an easy version of the problem. That is why the plan also includes a model that only sees the raw URL string (Notebook 03), which is the honest "hard mode" result.

Leakage Check 2: Too-Perfect Yes/No Features

If one class almost always has a flag and the other almost never does, that smells like an artifact of how the dataset was collected rather than a real-world rule. The standout is `IsHTTPS`: 100% of legitimate URLs use HTTPS, with zero exceptions, while about half the phishing URLs do too. A rule "no HTTPS = phishing" would never be wrong on this dataset, which is not true of the real web; it is a side effect of where the legitimate URLs were collected. `HasSocialNet`, `HasCopyrightInfo`, and `HasDescription` have huge gaps too (real sites have footers and meta tags, parked phishing pages usually do not). Dropping the top six giveaway flags (including `IsHTTPS`) and re-running the quick model barely changes accuracy. The easiness is spread across the whole feature set, not hidden in a few columns, so there is no clean line where we could keep "some" content features and get an honest problem. Decision: keep these features for the tabular models (Notebook 02) but call out `IsHTTPS` and friends in the report as collection artifacts, and

treat the URL-only model in Notebook 03 as the realistic difficulty level.

Leakage Check 3: Repeated Domains

`URL` and `Domain` are basically IDs, so they stay out of the feature set either way. But if the same domain shows up in both train and test, the model can memorize the domain instead of learning patterns. About 9% of rows share a domain with another row. The top of the list is telling: `ipfs.io`, `gateway.ipfs.io`, `cloudflare-ipfs.com` and `cf-ipfs.com` are all gateways to the same IPFS service, so exact string matching actually *under*-counts how related the rows are. Being stricter would mean grouping by registered domain (for example with the `tldextract` package), which Notebook 03 revisits. Comparing a normal random split against a split that keeps each domain on one side only shows essentially no difference: the score was not inflated by repeated domains, the dataset is just this easy. Still, the grouped split is the safer choice and costs nothing, so Notebook 02 uses `GroupShuffleSplit` on `Domain` and reports both.

The `TLD` Column

Two findings. First, 695 unique TLDs is too many to one-hot encode. Second, 599 of them are not TLDs at all: for URLs that use a raw IP address, whoever built the dataset took the last number of the IP as the "TLD" (for example, domain `198.98.58.123` gets TLD `123`). All 599 of those rows are phishing. `IsDomainIP` already captures this signal, so it is a data-quality quirk to mention, not something to fix. Instead of encoding `TLD` the plan is to use `TLDLegitimateProb`, a provided score for how trustworthy the TLD is. One worry: if UCI computed that score *from this dataset's own labels*, it would be target leakage just like `URLSimilarityIndex`. If the score had been computed from these labels, its correlation with the per-TLD share of legitimate rows would be close to 1; at about 0.18 it clearly was not, so it is fine to keep as a feature.

Which Safe Features Correlate with the Label?

The strongest safe features are mostly page-content ones (`HasSocialNet`, `HasCopyrightInfo`, `IsHTTPS`, and so on), matching the leakage-check results above

(Figure 3). One caveat: plain correlation understates skewed count features like `LineOfCode` or `NoOfImage`, so the notebook relies on model-based feature importance in Notebook 02 rather than this chart for feature selection.

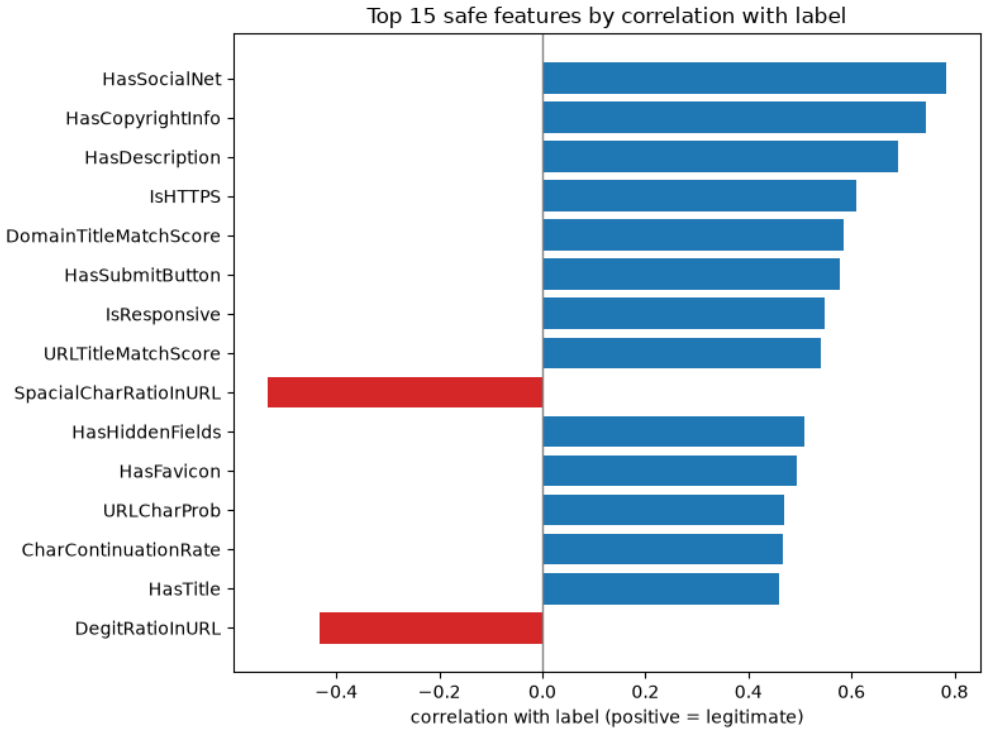

Figure 3

Top 15 safe features by absolute correlation with the label. Page-content indicators dominate; blue is positive and red is negative correlation.

A Closer Look at Individual Features

Everything above was about leakage. A normal EDA question is how the features actually differ between the two classes. The page-content features are the dramatic ones: the median phishing page has 12 lines of code, zero JS files, zero images, and zero external references, while the median legitimate page has over 1,100 lines of code. Most phishing pages are basically empty shells. The URL-shape differences are real but much smaller: phishing URLs are a bit longer and use more special characters. Figure 4 shows distributions for a few representative features (top row = URL features, bottom row = page-content features). The content features (bottom row) separate the classes almost by

themselves, with phishing piling up at zero, while the URL-shape features (top row) overlap a lot. This is another preview of why the URL-only model in Notebook 03 faces the harder problem.

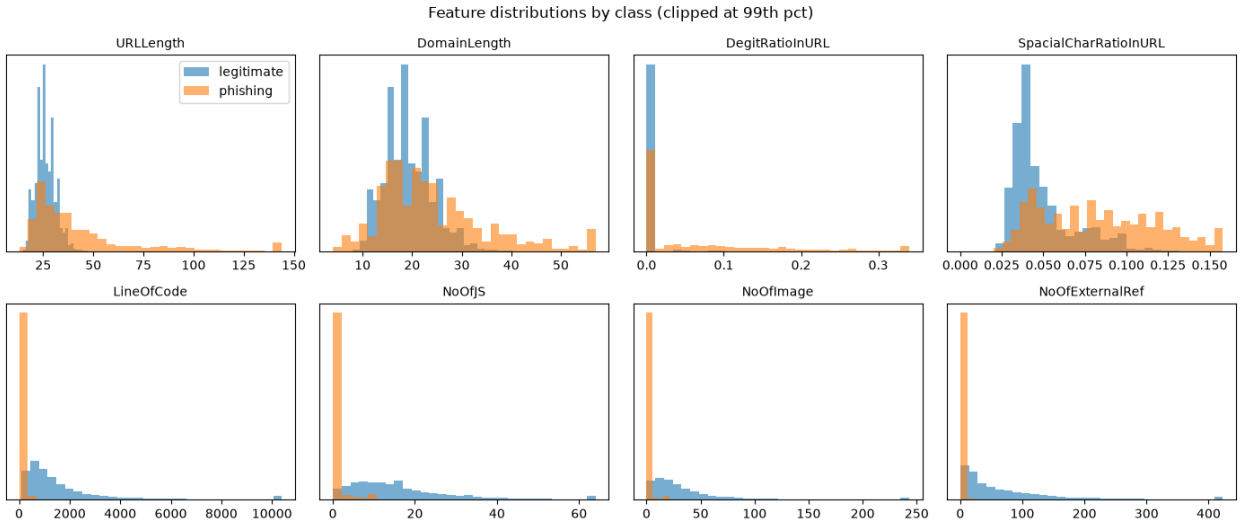

Figure 4

Per-class distributions (clipped at the 99th percentile) for representative URL-shape features (top) and page-content features (bottom). Phishing pages concentrate at zero on the content features.

Do Any Features Duplicate Each Other?

A lot of columns look like counts and ratios of the same thing. Strongly correlated features make logistic-regression coefficients hard to read and are a known risk for the SMOTE experiment. There is less redundancy than expected: only four pairs above an absolute correlation of 0.8, and only two near-duplicates (`DomainTitleMatchScore` / `URLTitleMatchScore` and `URLLength` / `NoOfLettersInURL`). Nothing so extreme that features need to be dropped up front. The heatmap (Figure 5) does show the strongest features are moderately correlated with each other; they all measure some version of "does this page have real content." So when Notebook 02 looks at feature importance, credit may be split between similar features and exact rankings should not be over-interpreted.

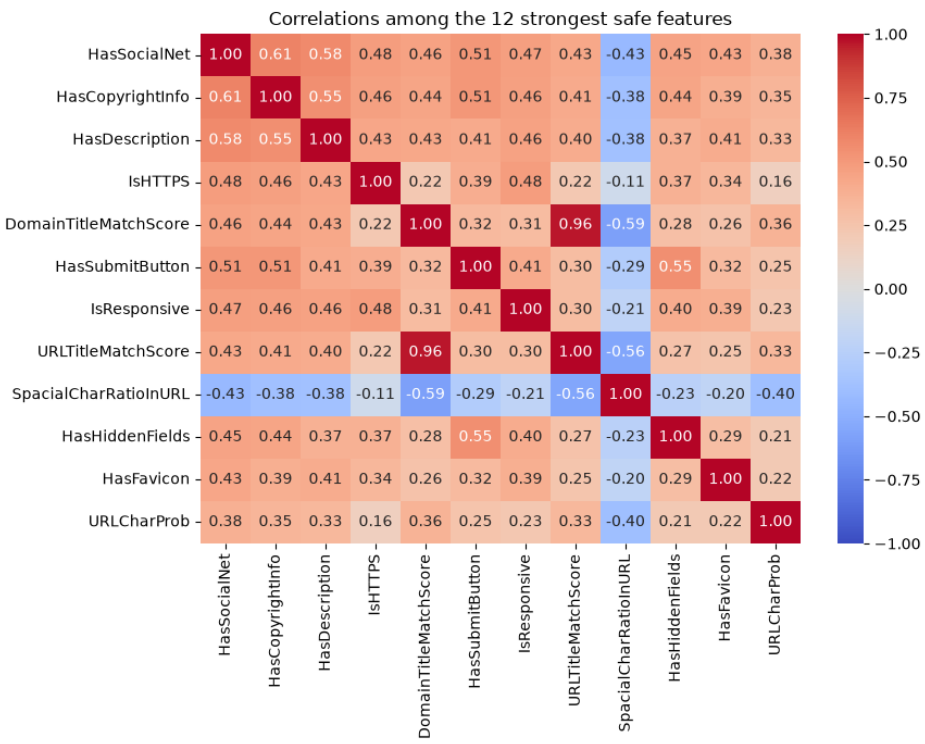

Figure 5

Correlation heatmap among the twelve features most correlated with the label. The strong content features are moderately inter-correlated.

Summary of the Data Audit

The audit establishes the following, all of which shape the modeling in Notebooks 02 and 03:

- **Class balance is mild** (57/43), not severe. Class weights first, SMOTE only as a comparison.
- **`URLSimilarityIndex` is leaky** (every legitimate row equals 100) and is dropped, but the other features still reach roughly 99.9%, so the dataset is easy with or without it. The URL-string-only model in Notebook 03 is the realistic-difficulty comparison.
- **`IsHTTPS` is a collection artifact** (100% of legitimate rows are HTTPS). It is kept but flagged; dropping the top giveaway flags barely changes accuracy.
- **425 duplicate URLs** lead Notebook 02 to drop URL duplicates before splitting.

- **9% of rows share a domain**, so Notebook 02 uses a domain-grouped split. The quick check says it does not change the score, but it is the safer setup.
- **599 IP-address rows have junk numeric "TLDs"** (all phishing), a data-quality quirk worth a sentence in the report.
- **`TLDLegitimateProb` checks out as non-leaky** (correlation about 0.18 with the labels), so it is used instead of one-hot encoding 695 TLDs.
- **Phishing pages are empty shells**: median 12 lines of code and zero JS/images/external references, versus 1,105 lines of code for the median legitimate page. URL-shape features overlap far more between the classes.
- **Feature redundancy is mild** (only four pairs with absolute correlation above 0.8), but the strong content features are moderately inter-correlated, so feature-importance rankings in Notebook 02 should not be over-interpreted.

The safe feature list is saved to `data/01_feature_columns.json` so the other notebooks all use the same agreed feature set.

Classical Models

This section reproduces the modeling and commentary of Notebook 02 (author: Jason Justice). Building on Notebook 01's leakage-clean feature list, it (1) engineers new features from the raw URL/Domain strings, (2) trains three classifiers on the clean feature set, (3) handles the mild class imbalance and tests whether SMOTE helps, (4) tunes the strongest model and reads off feature importance, (5) checks whether a handful of top features match the full model, and (6) adds a cost-sensitive decision layer. An important note on label orientation: the raw `label` is 1 for legitimate and 0 for phishing, and this notebook models **phishing as the positive class** (`y = 1 - label`) so that precision, recall, and PR-AUC all describe *catching phishing*.

Engineered Features

PhiUSIIL already ships lexical ratios and page-content counts. To add something genuinely new, the notebook builds four signals from the raw `URL` / `Domain` strings (held out of the clean set as identifiers) that those columns do not already encode:

- **`url_entropy` / `domain_entropy`**: character-level Shannon entropy. Randomly generated (DGA-style) domains tend to look more random.
- **`brand_distance`**: the smallest normalized edit distance from any significant domain label to a curated brand list. A domain that is *close but not equal* to a known brand is the classic typosquatting signal (for example, `paypa1`, `paypal-secure`).
- **`is_punycode`**: the domain uses `xn-` (IDN homograph attacks).
- **`is_shortener`**: the registered domain is a known URL shortener.

A quick look before modeling (medians by class and the distributions of the three continuous features) appears in Figure 6.

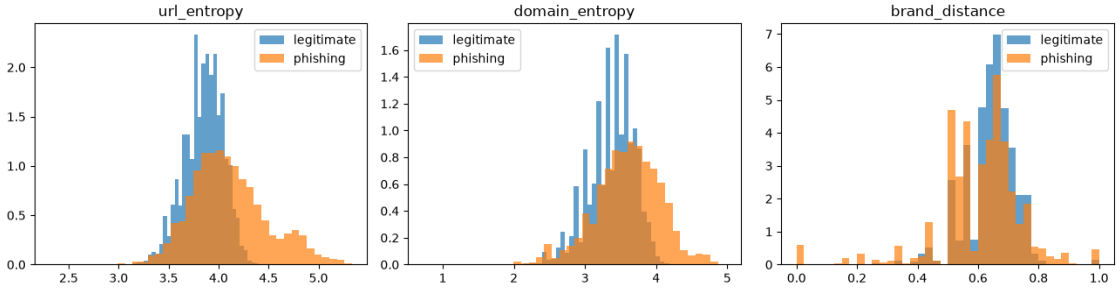

Figure 6

Distributions of the engineered continuous features by class. Class separation is modest, foreshadowing their low importance once the shipped content features are present.

Assembling the Modeling Frame

Following Notebook 01's manifest, the notebook drops duplicate URLs (so the same URL cannot land in both train and test), models phishing as the positive class, and splits with a domain-grouped 80/20 split so every row of a domain stays on one side.

Model Roster

Three classifiers are used, all treating phishing as the positive class: logistic regression (class-weighted) as an interpretable linear baseline; random forest (class-weighted) for a non-linear model with feature importance; and XGBoost (`scale_pos_weight`) as the strongest tabular model. The notebook reports accuracy, precision, recall, F1, ROC-AUC, and PR-AUC (`average_precision`), the last being the more informative summary under class imbalance. Results are in Table 1; confusion matrices are in Figure 7.

Table 1
Classical model performance on the shared test set (phishing = positive class).

Model	Accuracy	Precision	Recall	F1	ROC-AUC	PR-AUC
Logistic Regression	0.9995	0.9995	0.9993	0.9994	1.0	1.0
Random Forest	0.9999	0.9999	0.9998	0.9999	1.0	1.0
XGBoost	0.9999	1.0000	0.9998	0.9999	1.0	1.0

Class-Imbalance Handling

The imbalance here is mild (about 57/43), so the notebook starts with the cheap option, `scale_pos_weight`, and treats SMOTE as an explicit ablation rather than a default. Three setups are compared on the same XGBoost model: no balancing, class weighting, and SMOTE oversampling (Table 2). The three rows are nearly identical: on mild imbalance with an already-separable feature set, resampling buys essentially nothing (and SMOTE synthesizes points among correlated tabular features, a known overfitting risk). The notebook keeps `scale_pos_weight` as the cheap default.

Tuning the Strongest Model

`RandomizedSearchCV` over XGBoost is scored on PR-AUC (`average_precision`) and cross-validated with `StratifiedGroupKFold` so domains never straddle a fold boundary during tuning either.

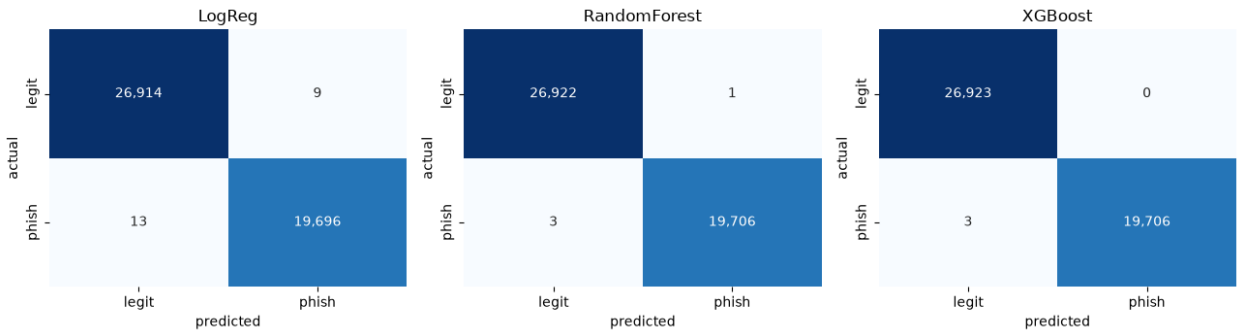

Figure 7

Confusion matrices for the three classifiers. Rows are the true class, columns the prediction, with phishing as the class of interest.

Feature Importance

The notebook uses two views: XGBoost's built-in gain importance (Figure 8) and model-agnostic permutation importance (the drop in PR-AUC when a feature is shuffled), computed on a test-set sample. It specifically checks where the four engineered features land.

Does a Handful of Features Match the Full Model?

The assignment calls out this pattern as the model of a good result: if a model using only the top-K features nearly matches the full model, that is an actionable finding, because a production filter could collect far fewer signals per URL. Retraining XGBoost on just the top eight features (`NoOfExternalRef`, `NoOfSelfRef`, `NoOfOtherSpecialCharsInURL`, `IsHTTPS`, `LineOfCode`, `HasSocialNet`, `NoOfSubDomain`, `URLLength`) nearly matches the tuned full model (Table 3).

Cost-Sensitive Decision Layer

Accuracy near the ceiling hides the real design question: where do you want the few remaining errors to fall? That depends on the cost of each error type, which is asymmetric for phishing. A false negative (phishing let through) risks credential theft or fraud, so its cost is set an order of magnitude higher, anchored loosely to public breach-cost figures. A false positive (legitimate site blocked) costs user friction and support load, an illustrative

Table 2
Imbalance-handling ablation on XGBoost.

Setup	Accuracy	Precision	Recall	F1	ROC-AUC	PR-AUC
none	0.9999	1.0	0.9998	0.9999	1.0	1.0
scale_pos_weight	0.9999	1.0	0.9998	0.9999	1.0	1.0
SMOTE	1.0000	1.0	0.9999	0.9999	1.0	1.0

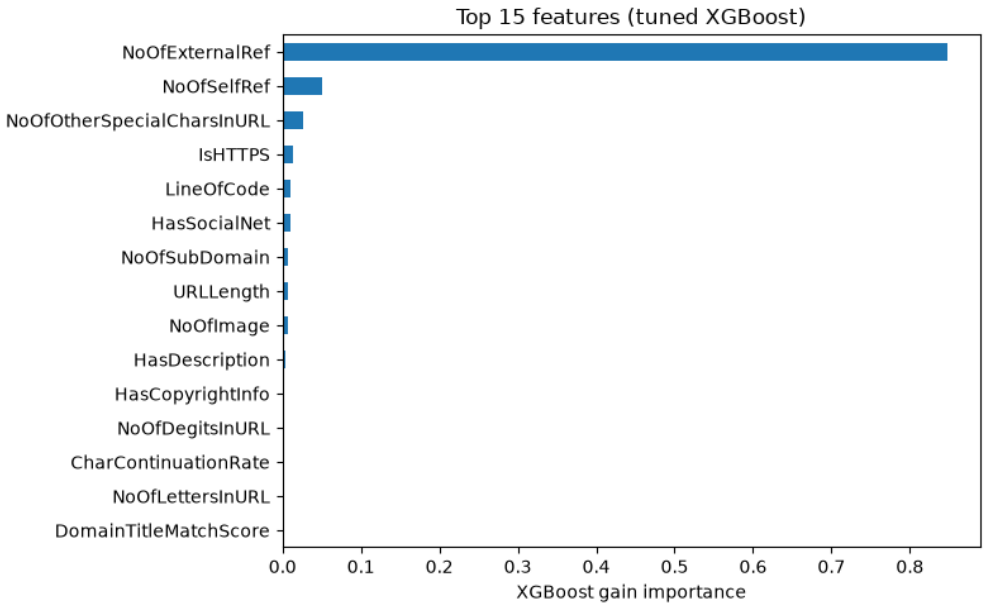

Figure 8
Top-15 XGBoost gain importances. Page-content and URL-structure columns dominate; the four engineered features do not appear in the top ranks.

Table 3
Full tuned model versus a top-8-feature model.

Model	Accuracy	Precision	Recall	F1	ROC-AUC	PR-AUC
XGBoost (tuned)	0.9999	1.000	0.9998	0.9999	1.0	1.0
XGBoost (top 8)	0.9994	0.999	0.9995	0.9993	1.0	1.0

estimate rather than a sourced figure. Sweeping the decision threshold and picking the operating point that minimizes expected cost per 1,000 URLs (Figure 9) yields a minimum-cost threshold of 0.24, at $21.44 per 1,000 URLs, versus $32.17 at the default threshold of 0.5.

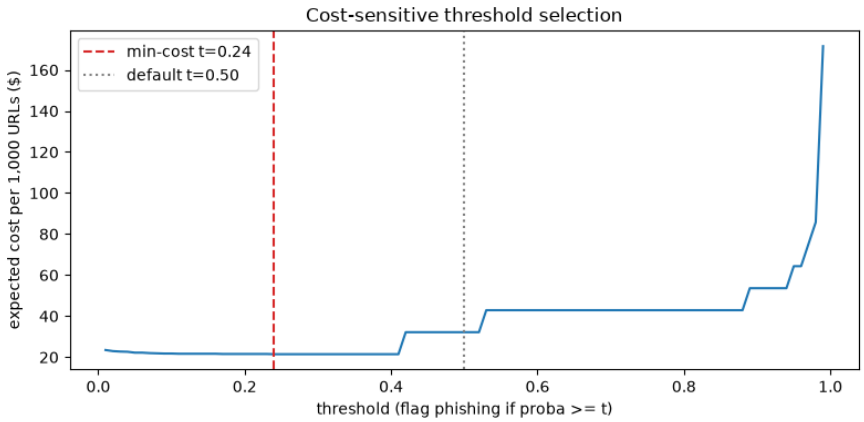

Figure 9

Expected cost per 1,000 URLs as a function of decision threshold. The minimum sits at 0.24, below the default 0.5, reflecting the higher cost assigned to false negatives.

Business Metric: It Depends on the Deployment

There is no single "right" metric; it follows the deployment scenario. A browser warning system (Google Safe Browsing style) is precision-first, because wrongly blocking a legitimate site is highly visible, so it accepts lower recall for very high precision. A mail or endpoint gateway that quarantines silently is recall-first, because a missed phishing email reaching an inbox is the expensive outcome. The notebook reports both operating points plus a three-tier allow/warn/block policy, which is closer to how real filters behave than one binary cutoff: a precision-first point at threshold 0.000 (precision 0.9900, recall 1.0000), a recall-first point at threshold 0.412 (precision 1.0000, recall 0.9999), and a three-tier policy that, on the test set, allows 26,926 URLs, blocks 19,704, and warns on 2.

Summary and Hand-Off

- **The engineered features barely move the needle here**: they land at the bottom of the importance ranking (ranks 22–54, about zero gain) and their by-class

medians overlap heavily. That is itself an honest result. The shipped page-content columns (`IsHTTPS`, `LineOfCode`, `NoOfExternalRef`, and so on) already carry essentially all the separating signal, so lexical entropy and brand-distance add little *on top of them*. They may still earn their keep in the URL-only regime (Notebook 03), where those content columns do not exist.

- **Mild imbalance needs only class weighting**; SMOTE added nothing here and risks overfitting correlated tabular features.
- **The problem is near-saturated on tabular features** (about 99.9%, matching Notebook 01 and the published PhiUSIIL numbers), so the interesting work is what to do with the tiny error budget: the cost-sensitive layer and the precision-first versus recall-first operating points make that an explicit, business-driven choice rather than a default 0.5.
- **A top-8-feature model nearly matches the full model**, an actionable result for a cheaper production filter.

The harder, more honest difficulty regime, a model that never sees the engineered or content features, is Notebook 03's character-level model on the raw URL string, alongside the clustering and the full state-of-the-art comparison.

Character-Level Model, Clustering, and State of the Art

This section reproduces Notebook 03 (author: Pranav Dhinakar). Following Notebook 02, phishing is modeled as the positive class so precision, recall, and PR-AUC describe catching phishing. Notebook 01 established the leakage-clean feature set and split conventions, and Notebook 02 built the tabular models and saved their metrics. This notebook (1) settles the train/test grouping convention and locks it to Notebook 02's for comparability, (2) trains a character-level CNN on the raw URL string only as the leakage-immune "hard mode" comparison, (3) clusters the phishing rows into attack "families," and (4) assembles the state-of-the-art comparison table.

Registered-Domain Grouping Decision

Notebook 01 split on the exact `Domain` string and flagged that this *under*-groups subdomains: `ipfs.io`, `gateway.ipfs.io` and `cf-ipfs.com` are the same service but count as three domains, so related rows can still land on both sides of a train/test split. The stricter alternative groups by registered domain (eTLD+1 via `tldextract`), which collapses those into one group. This matters because the character model, the clustering, and Notebook 02's tabular models must all use the *same* split, or the SOTA table compares numbers produced under different conditions. Three grouping strictnesses, run through the same quick logistic model, give the results in Table 4.

Table 4
Effect of grouping strictness on a quick logistic-regression score.

Grouping	Groups	Accuracy	F1
exact `Domain` (NB 01 / NB 02)	220,086	0.9995	0.9996
PaaS-aware eTLD+1 (PSL-private on)	197,709	0.9993	0.9994
naive eTLD+1	175,509	0.9990	0.9992

Accuracy is essentially invariant (0.9990–0.9995) across a 44,577-group range of strictness. The drop is monotonic with strictness, the direction leakage would predict, but negligible in size, reinforcing Notebook 01's finding that the dataset is easy regardless of how conservatively the split is drawn. Because the score is grouping-invariant, the notebook **adopts Notebook 02's exact `Domain` grouping** (`GroupShuffleSplit`, `test_size=0.2`, `random_state=42`) rather than reworking it. This makes the character model share Notebook 02's exact test set, so the SOTA table compares numbers produced under identical conditions. The stricter registered-domain analysis is retained as evidence that this choice does not inflate the results.

Character-Level Model on the Raw URL

The tabular models in Notebook 02 use page-content and lexical features that make PhiUSIIL trivially easy (about 99.9%). This model sees only the raw URL string: no

engineered features, no page content, no `URLSimilarityIndex`. It cannot leak, because it has access to nothing but the characters of the URL itself, so it is the leakage-immune comparison point for the URL-only lexical literature (Sahingoz et al. (2019) reached 97.98% with an NLP-feature random forest). Each URL is tokenized at the character level, embedded, and passed through a small 1-D CNN with a sigmoid output, trained with class weights on the same exact-`Domain` split used in Notebook 02 so the numbers are comparable.

Architecture

The model is a compact character-level CNN (Kim (2014), adapted to characters). An embedding maps each character id to a 48-dimensional vector (`padding_idx=0`). Three parallel 1-D convolutions of kernel widths 3, 5, and 7 (128 filters each) read character 3-, 5-, and 7-grams. A global max-pool over position is followed by concatenation of the three widths, dropout (0.4), a linear layer, and a single logit. The loss is `BCEWithLogitsLoss` with `pos_weight = n_neg/n_pos` for class weighting. Inputs use `MAX_LEN = 200`, which covers roughly the 99th percentile of URL lengths, with longer URLs truncated. A grouped 10% validation slice (domains do not straddle train/val) is used only to monitor convergence and keep the best-PR-AUC epoch; the test set is untouched until evaluation.

Findings

The model converged in one epoch (validation PR-AUC 0.9988 at epoch 1). On the held-out test set its metrics essentially tie the tuned tabular model (Table 5). At threshold 0.5 it makes 1 false positive and 69 false negatives on 46,632 test URLs (Figure 10).

Table 5
CharCNN (URL only) versus tuned XGBoost (Notebook 02).

Metric	CharCNN (URL only)	tuned XGBoost (NB 02)
accuracy	0.9985	∼0.999
precision (phishing)	0.9999	∼0.999
recall (phishing)	0.9965	∼0.999
PR-AUC	0.9993	∼0.999

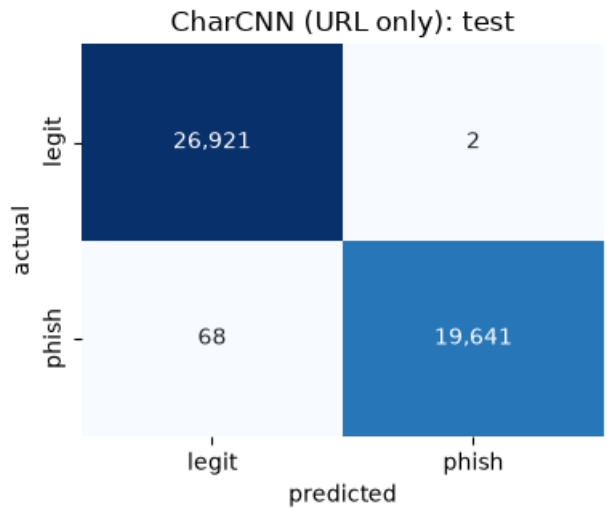

Figure 10

CharCNN confusion matrix on the 46,632-row test set: 1 false positive and 69 false negatives at threshold 0.5.

The URL-only model does not come in harder than the tabular models. It essentially ties them and lands above the URL-only lexical literature (Sahingoz et al. (2019): 97.98%; the char-CNN band in the SOTA comparison: 97–99%). So PhiUSIIL's separability is not confined to the page-content columns Notebook 01 flagged; it is present in the raw URL strings themselves. Legitimate and phishing URLs here come from systematically different sources (well-known registrable domains versus IP-address hosts, IPFS gateways, PaaS subdomains, randomized tokens), differences visible character by character. This is the project's own evidence for the benchmark-leakage critique in the SOTA comparison (Dalton et al., 2025; Das et al., 2019).

Two caveats apply to the SOTA table. First, this model beats the URL-only literature because PhiUSIIL is more separable than the cross-source corpora those papers used, not because the architecture is stronger; the claim is "about 99.85% on PhiUSIIL," not a general result. Second, the exact-`Domain` split still lets registered-domain siblings on shared platforms (`web.app`, `firebaseapp.com`, `repl.co`) fall on both sides, which a character model could exploit as platform-level memorization. The grouping analysis showed the tabular score is near-invariant to this (0.9993 PaaS-aware versus 0.9995 exact),

so it is unlikely to be the main driver, but the CNN is more exposed to it than logistic regression.

Clustering: Phishing Attack Families

The models so far are supervised. This section adds an unsupervised method, the project's second algorithm type, and asks a different question: not whether a URL is phishing, but what kinds of phishing are in the data. The notebook clusters the phishing rows only on Notebook 01's leakage-clean feature set (label held out) and profiles the groups as attack "families." This is standard in the phishing literature (Feng et al., 2022 clustered phishing kits; Althobaiti et al., 2023 clustered campaigns; Zia and Kalidass, 2025 clustered lexical URL tokens). Cluster IDs are not fed back into the classifier, to avoid a second leakage-review cycle. The method is KMeans with k chosen by elbow and silhouette, centroid profiling to name the families, PCA for a 2-D view, and HDBSCAN as a density-based cross-check.

Choosing k

The elbow and silhouette scans over k appear in Figure 11 and Table 6. Here $k = 2$ has by far the highest silhouette (0.44): the strongest structure is a single binary split, matching Notebook 01's finding that phishing pages divide into content-less shells versus content-bearing pages. The elbow is otherwise soft, typical of high-dimensional tabular data with correlated features. Because $k = 2$ is too coarse for a families deliverable, the notebook uses $k = 5$, the silhouette peak among $k \geq 3$ and where the elbow flattens. It reports the $k = 2$ split as the dominant coarse structure and profiles $k = 5$ as interpretable strata. The silhouettes at $k = 5$ (about 0.15) are low, so these are overlapping descriptive families rather than crisply separated clusters; HDBSCAN checks how much crisp density structure exists.

The Five Families

Each cluster's raw-unit means and its centroid's top distinguishing features (in standard deviations from the phishing-wide mean) give the families in Table 7.

Table 6
KMeans model-selection scan over k (silhouette on a 10k sample).

k	Inertia	Silhouette (10k)
2	4,500,166	0.437
3	4,248,341	0.130
4	3,923,595	0.149
5	3,736,818	0.154
6–10	3.58M–3.07M	0.089–0.121

Empty-shell pages dominate at 68%, consistent with Notebook 01's finding that most phishing rows are content-less and with the $k = 2$ super-split (87/13) that separates these from content-bearing pages. The two active attack types separate cleanly: credential-harvesting forms (c3), with the password/submit/hidden-field signature of a fake login, and HTTP lookalike stubs (c4), which look brand-consistent but lack HTTPS. Cluster c1 holds only 5 rows; KMeans minimizes squared distance and is sensitive to outliers, so a few 4,000-character URLs captured their own centroid. This is a KMeans artifact, not a real family, and the HDBSCAN cross-check relabels these as noise.

What the Clustering Shows

KMeans ($k = 5$) partitions phishing into interpretable strata: one dominant empty-shell family (68%) and two distinct active types, credential-harvesting forms (12%, password/submit/hidden-field signature) and HTTP lookalike stubs (19%, domain-matching titles but no HTTPS), plus a small IP-host / obfuscated-URL group (1%) and a 5-row outlier artifact. This mirrors the supervised EDA in Notebook 01: most phishing pages are content-less shells, and the rest split into "steal credentials directly" versus "look legitimate cheaply" (Figure 12).

The structure is soft. Three signals say these are convenient strata over a partly continuous space rather than sharply separated kinds. First, silhouettes are low (about 0.15 at $k = 5$; only the $k = 2$ empty-versus- content split scores well, at 0.44). Second, PCA is diffuse: the first two components explain 22.9% of variance, and the projection is

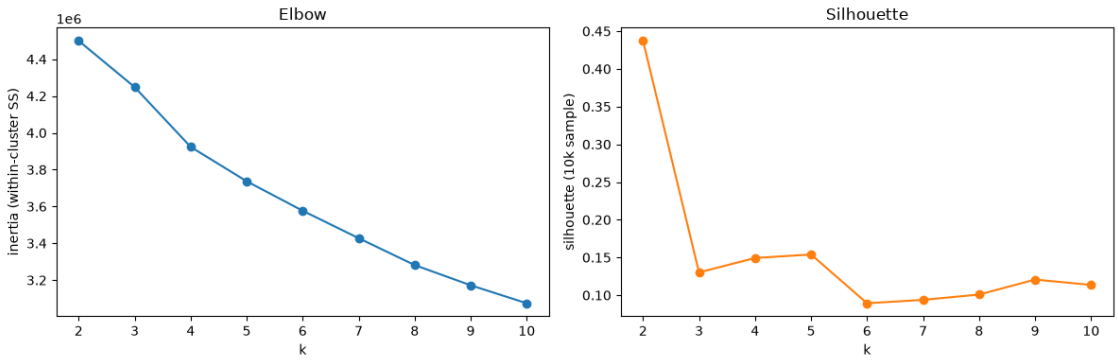

Figure 11

Elbow (inertia) and silhouette curves for KMeans. The silhouette peaks sharply at $k = 2$; $k = 5$ is the local peak used for interpretable families.

one dense overlapping mass with a few outlier spikes (the 5 long-URL points, one IP-host page, the credential-form tail), not five islands. Third, HDBSCAN found 25 micro-clusters and left 30% as noise on a 25k sample, rather than five clean families. The micro-clusters likely correspond to near-identical phishing kits (templated pages reused across many URLs), the finer level Feng et al. (2022) found; the 30% is genuinely continuous. The 5-row KMeans outlier does not survive as an HDBSCAN cluster, confirming it was an artifact. So KMeans $k = 5$ is the descriptive lens that names recognizable attack families and ties them to the supervised findings, while HDBSCAN is the validity check showing the true structure is a few dominant modes plus many small kit-level pockets and a continuous remainder. The contrast between partition-based and density-based clustering is itself a result for the write-up.

State-of-the-Art Comparison

Two framing rules govern the comparison. First, numbers are comparable only within a dataset; accuracy on PhiUSIIL and accuracy on another corpus are different measurements, so the CharCNN's about 99.85% is on PhiUSIIL, directly comparable to other PhiUSIIL numbers and only contextually comparable to URL-only models on other corpora. Second, published numbers are flagged by verification status: several come from secondary citations because the primary papers were not directly accessible, and sources

Table 7
The five phishing families from KMeans ($k = 5$).

Cluster (size)	Attack family	Defining signature
c0 – 68,743 (68.4%)	Empty-shell pages	No title, no forms, about 40 lines of code, low title-match (parked/stub, no real content)
c3 – 11,809 (11.7%)	Credential-harvesting forms	Password, submit, hidden fields, images, CSS, favicon, about 267 lines of code (fake logins)
c4 – 18,830 (18.7%)	HTTP lookalike stubs	Title matches domain, short clean URLs, only 17% HTTPS (brand-consistent, plain HTTP)
c2 – 1,133 (1.1%)	IP-host / obfuscated pages	`HasObfuscation`, 39% IP-hosted, long URLs, many `?` params
c1 – 5 (0.0%)	Outlier micro-cluster	URL length about 4,128 chars, extreme obfuscated/digit/ampersand counts (ultra-long parameter-stuffed URLs)

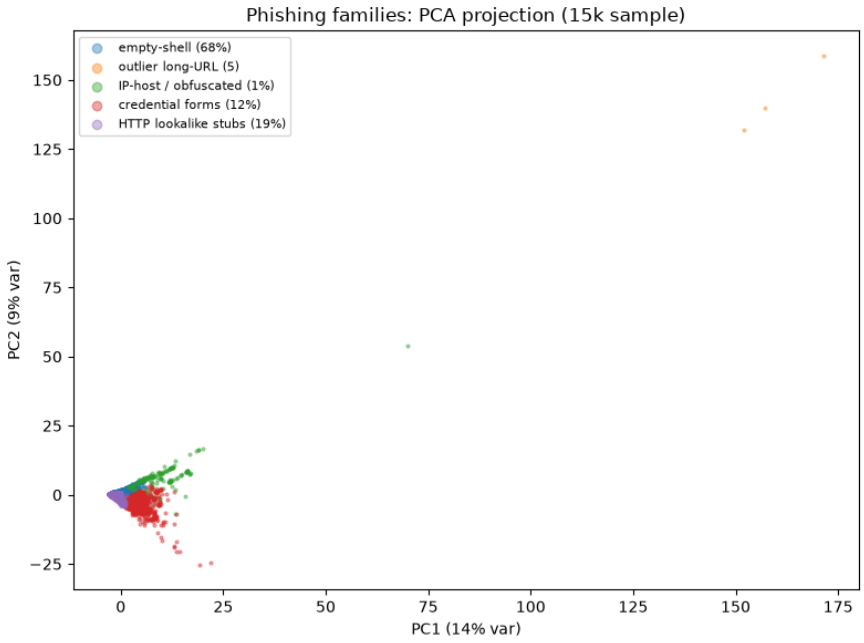

Figure 12
PCA projection of the phishing families (15k sample). The projection is one dense overlapping mass with a few outlier spikes rather than five separated islands; the first two components explain 22.9% of variance.

sometimes disagree, so those are marked unverified and treated as directional. Figure 13 places the project's supervised models against the published PhiUSIIL band.

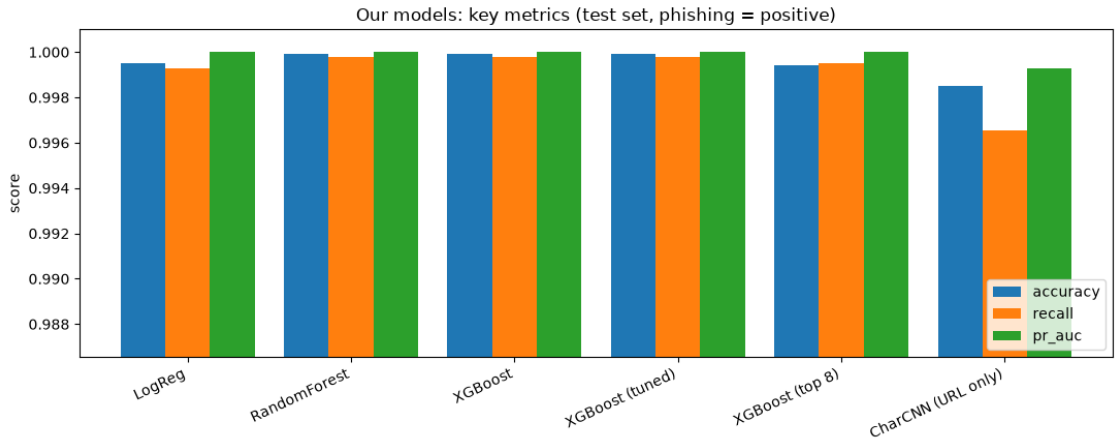

Figure 13

The project's models against the published PhiUSIIL 99.5–100% band. All five supervised models fall inside the band; the CharCNN is the informative near-tie.

Published Results in Context

Table 8 lists results on the same dataset (PhiUSIIL), directly comparable to the project's rows, and Table 9 lists URL-only and lexical results on other datasets, included for context only.

These bracket a 97–99% range for URL-only models on harder cross-source corpora. The CharCNN lands above that band because PhiUSIIL is more separable at the raw-string level, not because the architecture is stronger. Precedents for the other methods: for the imbalance ablation, Omari et al. (2025) report SMOTE-NC + XGBoost at 98.0% precision / 98.5% recall; for the clustering, Feng et al. (2022) (k-medoid on phishing kits, 90.1% TPR) and Zia and Kalidass (2025) (LSH on lexical tokens, 93% on PhishStorm) frame attack-family clustering as standard practice.

Where Our Results Land

Against PhiUSIIL, every one of the project's models sits in the published 99.5–100% band: logistic regression 0.9995, random forest 0.9999, XGBoost 1.0000, tuned XGBoost 0.9999, and a top-8-feature model at 0.9993. This reproduces the dataset authors' range

Table 8
Published results on PhiUSIIL, with the project's own rows.

Study	Approach	Reported / status
Prasad and Chandra (2024) (dataset authors)	similarity-index + incremental learning	∼99.2–99.8% (one source 99.97%); unverified, sources disagree
Pentapalli et al. (2025)	TSK fuzzy; excluded `URLSimilarityIndex`	qualitative agreement with our leakage call
Independent reproductions (Kaggle, small papers)	RF / DT / KNN / LR / XGB / FCNN / BERT	99.5–100% band; aggregated, unverified
This project, tuned XGBoost (NB 02)	full leakage-clean feature set	see Table 3
This project, CharCNN (URL only)	raw URL characters only	0.9985 acc / 0.9993 PR-AUC

and the independent reproductions across model families; it confirms a known ceiling rather than producing an outlier. At four decimals almost everything rounds to about 1.0, so accuracy ordering is noise. The informative comparison is the CharCNN: 0.9985 accuracy and 0.9993 PR-AUC on the same 46,632-row test split, within 0.0015 accuracy of the full-feature XGBoost. A model denied `URLSimilarityIndex`, the page-content flags, and the engineered features, seeing only the raw URL string, matches models that see all 49 clean features. That is direct evidence from the project's own pipeline that PhiUSIIL's separability is in the URL strings, not just the content columns Notebook 01 flagged, and it corroborates the benchmark-leakage critique rather than only citing it. The supervised problem is saturated; the useful question is why. The contribution of this notebook is not a new ceiling number but the leakage-immune CharCNN and the clustering, which together characterize what makes the benchmark easy and what kinds of phishing it contains.

Synthesis and Discussion

The three notebooks converge on one claim: on PhiUSIIL, the separability of phishing from legitimate URLs is intrinsic to the data, not an artifact of a single leaky column or a favorable split. The project demonstrates this three separate ways. The data audit showed that removing the obvious leak, `URLSimilarityIndex`, leaves the remaining

Table 9
URL-only / lexical literature on other datasets (context only).

Study	Approach	Reported acc	Status
Sahingoz et al. (2019)	RF on 39 NLP lexical features	97.98%	corroborated; firmest external number
Yerima and Alzaylaee (2020)	char-CNN	98.2% (F1 0.976)	reported (arXiv)
Maneriker et al. (2021) (URLTran)	BERT on subword URLs	about 96%	reported (arXiv)
MiniLM-CNN-LSTM authors (2024)	MiniLM-CNN-LSTM hybrid	98.98%	reported
Dubey et al. (2025)	char-CNN + LightGBM	99.82%	reported (arXiv)
Mohammad et al. (2014)	self-structuring NN	92–96% (disputed)	unverified; hedge or drop

features at about 99.9%. The grouping analysis showed that tightening the train/test split from exact-domain to registered-domain grouping, across a range of more than 44,000 groups, moves accuracy only from 0.9995 to 0.9990. And the character model, which sees nothing but the raw URL string and therefore cannot leak, still reached 0.9985 accuracy. Three different attempts to make the problem harder all failed to move the number in any meaningful way. That consistency is the result.

The project also satisfies the course requirement to combine at least two algorithm families under experimental comparison. It uses supervised classification (logistic regression, random forest, and XGBoost) and unsupervised clustering (KMeans and HDBSCAN), and it runs several controlled comparisons within those: an imbalance ablation (none versus class weighting versus SMOTE), hyperparameter tuning with grouped cross-validation, a full-model versus top-8-feature comparison, a partition-based versus density-based clustering cross-check, and a character-level model set against both the tabular models and the published literature. The empirical comparisons are presented graphically where possible, as the assignment prefers.

Two things are worth stating about what the project does and does not claim. It does claim that PhiUSIIL is easy for reasons that go all the way down to the raw URL characters, and that the easy-benchmark result should be read as evidence about the

dataset rather than about any one model. It does not claim a general phishing-detection breakthrough. The character model beats the URL-only literature because PhiUSIIL is more separable than the cross-source corpora those studies used, not because the architecture is stronger. The honest phrasing is "about 99.85% on PhiUSIIL," full stop.

A consistent caveat runs through the state-of-the-art comparison: many of the published numbers are vendor-reported or come from secondary citations that were not independently verified, and sources sometimes disagree. Those numbers are treated as directional rather than exact, and the tables flag their verification status. The safest reading of the comparison is that the project's results reproduce a known ceiling, not that any single external figure is authoritative.

The main limitations follow from the benchmark itself. Because PhiUSIIL is so separable, the accuracy ordering among models is noise, so the useful work shifted to interpretation: the cost-sensitive operating points, the feature-importance finding that a handful of features suffices, and the clustering that names attack families. The exact-domain split also leaves platform-level siblings on shared hosting on both sides of the split, which the character model is more exposed to than the linear model; the grouping analysis suggests this is not the main driver, but a fully platform-aware split would close the gap. Natural next steps are to test the trained models on a different corpus to measure how much of the performance is PhiUSIIL-specific, and to feed the kit-level structure that HDBSCAN surfaced into a more operational grouping of phishing campaigns.

Conclusion

This project built and compared a full phishing-URL detection pipeline on PhiUSIIL: a leakage audit and exploratory analysis, three classical classifiers with imbalance, tuning, feature-importance and cost-sensitive analyses, a character-level convolutional model on the raw URL string, and an unsupervised clustering of phishing into attack families. Every supervised model landed in the published 99.5–100% band, and the leakage-immune character model essentially tied them. The contribution is not a new

accuracy number but an explanation of why the benchmark is easy, established from three independent directions, together with a characterization of the kinds of phishing the dataset contains. The result is a report that treats near-perfect accuracy as something to be explained rather than celebrated.

References

Althobaiti et al. (2023). *Clustering phishing campaigns* [VERIFY full citation (authors, venue).].

Dalton et al. (2025). *On benchmark leakage in phishing detection* [VERIFY full citation (authors, venue).].

Das, A., Baki, S., El Aassal, A., Verma, R., & Dunbar, A. (2019). SoK: A comprehensive reexamination of phishing research from the security perspective [VERIFY volume/pages/DOI.]. *IEEE Communications Surveys & Tutorials*.

Dubey et al. (2025). *Character-CNN with LightGBM for phishing detection* [VERIFY full author list and title.]. arXiv: 2512.16717.

Feng et al. (2022). Clustering phishing kits [VERIFY author list and title.]. *PeerJ Computer Science*, *8*, e868.

Kim, Y. (2014). Convolutional neural networks for sentence classification [arXiv:1408.5882]. *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 1746–1751.

Maneriker, P., et al. (2021). *URLTran: Improving phishing URL detection using transformers* [VERIFY full author list.]. arXiv: 2106.05256.

MiniLM-CNN-LSTM authors. (2024). A MiniLM-CNN-LSTM hybrid for phishing URL detection [MDPI. VERIFY authors, volume, pages.]. *Technologies*.

Mohammad, R. M., Thabtah, F., & McCluskey, L. (2014). Predicting phishing websites based on self-structuring neural network. *Neural Computing and Applications*, *25*(2), 443–458.

Omari et al. (2025). SMOTE-NC with XGBoost for phishing detection [VERIFY full author list, title, pages.]. *International Journal of Advanced Computer Science and Applications*, *16*(2).

Pentapalli et al. (2025). *Phishing detection using a TSK fuzzy classifier* [VERIFY full author list and title.]. arXiv: 2504.18636.

Prasad, A., & Chandra, S. (2024). PhiUSIIL: A diverse security profile empowered phishing URL detection framework based on similarity index and incremental learning [VERIFY exact page/DOI. Dataset also at UCI ML Repository, id 967.]. *Computers & Security*, *136*, 103545.

Sahingoz, O. K., Buber, E., Demir, O., & Diri, B. (2019). Machine learning based phishing detection from URLs. *Expert Systems with Applications*, *117*, 345–357.

Yerima, S. Y., & Alzaylaee, M. K. (2020). *High accuracy phishing detection based on convolutional neural networks* [VERIFY exact title/venue.]. arXiv: 2004.03960.

Zia & Kalidass. (2025). *Locality-sensitive hashing of lexical URL tokens* [VERIFY full author list and title.]. arXiv: 2502.13171.

Appendix A

Contribution Statement

The project was divided by notebook, with each member owning one notebook end to end and the report assembled collaboratively.

- **Guna Pasupathy** authored Notebook 01 (data loading, the three-part leakage audit, exploratory data analysis, and the saved leakage-clean feature manifest that the other notebooks consume).
- **Jason Justice** authored Notebook 02 (feature engineering from the raw URL/Domain strings, the three classical classifiers, the class-imbalance ablation, hyperparameter tuning, feature-importance analysis, the top-8-feature comparison, and the cost-sensitive decision layer).
- **Pranav Dhinakar** authored Notebook 03 (the registered-domain grouping decision, the character-level CNN on raw URLs, the KMeans/HDBSCAN clustering of attack families, and the state-of-the-art comparison) and assembled this report.

[Team: confirm the exact split of report writing, presentation, and repository maintenance here before submission.]

Appendix B

Generative-AI Use Disclosure

Consistent with the course policy, the team discloses its use of generative-AI tools. All modeling code, experimental design, analysis, and numerical results in the three notebooks are the team's own work. A large-language-model assistant was used to help assemble and format this LaTeX report from the notebooks' existing author-written markdown notes, to convert notebook tables and figures into report tables and captions, and to draft the introduction, synthesis, and conclusion sections, which were then reviewed and edited by the authors. The assistant did not generate the analysis or the results. Any AI-assisted passages were checked against the notebook outputs for accuracy. *[Team: add any other tools used during development, for example code assistance in the notebooks, and adjust this statement to match.]*

Appendix C

Source Code

The complete, documented source code for the three notebooks is listed below, exported from the committed Jupyter notebooks. The authoritative version, with full commit history and outputs, is in the team GitHub repository linked on the title page.

Notebook 01 – Load, Leakage, EDA (Guna Pasupathy)

```
# Notebook 01 - Load, Leakage, EDA (Guna Pasupathy)
# Exported from 01_load_leakage_eda.ipynb (code cells only, in order).

# ===== Code cell 1 ======================================
import json
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, f1_score
from sklearn.model_selection import GroupShuffleSplit, train_test_split
from sklearn.preprocessing import StandardScaler

DATA_PATH = Path("../data/phiusiil.csv")

# ===== Code cell 2 ======================================
df = pd.read_csv(DATA_PATH)
y = df["label"]

print(f"shape: {df.shape}")
print(f"missing values: {df.isna().sum().sum()}")
print()
print("column types:")
print(df.dtypes.value_counts())
df.head(3)

# ===== Code cell 3 ======================================
counts = y.value_counts().sort_index()

plt.figure(figsize=(5, 4))
plt.bar(
    ["phishing (0)", "legitimate (1)"],
    counts.values,
    color=["tab:orange", "tab:blue"],
)
for i, c in enumerate(counts.values):
    plt.text(i, c + 2000, f"{c:,} ({c / len(df):.1%})", ha="center")
plt.ylabel("rows")
plt.title("Class balance")
```

```
plt.ylim(0, counts.max() * 1.15)
plt.tight_layout()
plt.show()

# ===== Code cell 4 ======================================
dup_url_extra = int(df["URL"].duplicated().sum())
dups = df[df["URL"].duplicated(keep=False)]
conflicts = dups.groupby("URL")["label"].nunique()

print(f"exact duplicate rows: {df.duplicated().sum()}")
print(f"repeated URLs (extra rows): {dup_url_extra}")
print(f"repeated URLs with conflicting labels: {(conflicts > 1).sum()}")

# ===== Code cell 5 ======================================
print(df.groupby("label")["URLSimilarityIndex"].describe())

n_100 = ((df["URLSimilarityIndex"] == 100) & (y == 0)).sum()
print(f"\nphishing rows that also score exactly 100: {n_100}")

# ===== Code cell 6 ======================================
bins = np.linspace(0, 100, 41)

plt.figure(figsize=(8, 4))
plt.hist(
    df.loc[y == 1, "URLSimilarityIndex"],
    bins=bins,
    alpha=0.7,
    color="tab:blue",
    label="legitimate (1)",
)
plt.hist(
    df.loc[y == 0, "URLSimilarityIndex"],
    bins=bins,
    alpha=0.7,
    color="tab:orange",
    label="phishing (0)",
)
plt.yscale("log")
plt.xlabel("URLSimilarityIndex")
plt.ylabel("count (log scale)")
plt.title("URLSimilarityIndex by class")
plt.legend()
plt.tight_layout()
plt.show()

# ===== Code cell 7 ======================================
def quick_logreg(X, y, groups=None):
    """Train a basic logistic regression, return test acc and F1."""
    if groups is None:
        X_train, X_test, y_train, y_test = train_test_split(
            X, y, test_size=0.2, random_state=42, stratify=y
        )
    else:
        # keep all rows of the same group on one side of the split
```

```
        gss = GroupShuffleSplit(n_splits=1, test_size=0.2, random_state=42)
        train_idx, test_idx = next(gss.split(X, y, groups=groups))
        X_train, X_test = X.iloc[train_idx], X.iloc[test_idx]
        y_train, y_test = y.iloc[train_idx], y.iloc[test_idx]

    scaler = StandardScaler()
    X_train = scaler.fit_transform(X_train)
    X_test = scaler.transform(X_test)
    model = LogisticRegression(max_iter=1000)
    model.fit(X_train, y_train)
    preds = model.predict(X_test)
    return accuracy_score(y_test, preds), f1_score(y_test, preds)


numeric_cols = df.select_dtypes("number").columns
safe_cols = [
    c for c in numeric_cols if c not in ("URLSimilarityIndex", "label")
]

acc1, f11 = quick_logreg(df[["URLSimilarityIndex"]], y)
acc2, f12 = quick_logreg(df[safe_cols], y)

print(f"URLSimilarityIndex alone: acc={acc1:.4f}  f1={f11:.4f}")
print(f"all other numeric cols:   acc={acc2:.4f}  f1={f12:.4f}")

# ===== Code cell 8 ========================================
binary_cols = [
    c for c in numeric_cols if c != "label" and set(df[c].unique()) <= {0, 1}
]

rates = df.groupby("label")[binary_cols].mean().T
rates.columns = ["phishing_rate", "legit_rate"]
rates["gap"] = (rates["legit_rate"] - rates["phishing_rate"]).abs()
rates = rates.sort_values("gap", ascending=False)

print(f"{len(binary_cols)} binary feature columns")
rates.head(10).round(3)

# ===== Code cell 9 ========================================
giveaways = rates.index[:6].tolist()
print("dropping:", giveaways)

acc3, f13 = quick_logreg(df[[c for c in safe_cols if c not in giveaways]], y)
print(f"without giveaway flags: acc={acc3:.4f}  f1={f13:.4f}")

# ===== Code cell 10 =======================================
domain_counts = df["Domain"].value_counts()
repeated = domain_counts[domain_counts > 1]

print(f"unique domains: {len(domain_counts):,} / {len(df):,} rows")
print(
    f"domains appearing more than once: {len(repeated):,} "
    f"({repeated.sum():,} rows, {repeated.sum() / len(df):.1%})"
)
```

```
domain_counts.head(10)

# ===== Code cell 11 ========================================
acc4, f14 = quick_logreg(df[safe_cols], y, groups=df["Domain"])

print(f"normal random split:  acc={acc2:.4f}  f1={f12:.4f}")
print(f"domain-grouped split: acc={acc4:.4f}  f1={f14:.4f}")

# ===== Code cell 12 ========================================
print(f"unique TLDs: {df['TLD'].nunique()}")
print(df["TLD"].value_counts().head(10))

weird = df["TLD"].astype(str).str.fullmatch(r"\d+")
is_ip = df.loc[weird, "IsDomainIP"] == 1
print(f"\nrows where the 'TLD' is just a number: {weird.sum()}")
print(f"of those, IsDomainIP == 1: {is_ip.sum()}")
print(f"labels of those rows: {df.loc[weird, 'label'].unique()}")
df.loc[weird, ["Domain", "TLD", "label"]].head(3)

# ===== Code cell 13 ========================================
legit_share_per_tld = df.groupby("TLD")["label"].mean()
legit_share = df["TLD"].map(legit_share_per_tld)

corr = legit_share.corr(df["TLDLegitimateProb"])
print(f"corr with per-TLD legit share in this data: {corr:.3f}")

# ===== Code cell 14 ========================================
corrs = df[safe_cols].corrwith(y)
top15 = corrs.reindex(corrs.abs().sort_values().index).tail(15)
colors = ["tab:blue" if v > 0 else "tab:red" for v in top15]

plt.figure(figsize=(8, 6))
plt.barh(top15.index, top15.values, color=colors)
plt.axvline(0, color="gray", linewidth=1)
plt.xlabel("correlation with label (positive = legitimate)")
plt.title("Top 15 safe features by correlation with label")
plt.tight_layout()
plt.show()

# ===== Code cell 15 ========================================
non_binary = [c for c in safe_cols if c not in binary_cols]

medians = df.groupby("label")[non_binary].median().T
medians.columns = ["phishing_median", "legit_median"]
print(f"{len(non_binary)} non-binary numeric features")
medians.round(3)

# ===== Code cell 16 ========================================
feats = [
    "URLLength",
    "DomainLength",
    "DegitRatioInURL",
    "SpacialCharRatioInURL",
    "LineOfCode",
```

```
    "NoOfJS",
    "NoOfImage",
    "NoOfExternalRef",
]

fig, axes = plt.subplots(2, 4, figsize=(14, 6))
for ax, col in zip(axes.flat, feats):
    # clip extreme outliers so the plots stay readable
    clipped = df[col].clip(upper=df[col].quantile(0.99))
    ax.hist(
        clipped[y == 1],
        bins=30,
        alpha=0.6,
        color="tab:blue",
        density=True,
        label="legitimate",
    )
    ax.hist(
        clipped[y == 0],
        bins=30,
        alpha=0.6,
        color="tab:orange",
        density=True,
        label="phishing",
    )
    ax.set_title(col, fontsize=10)
    ax.set_yticks([])
axes[0, 0].legend()
fig.suptitle("Feature distributions by class (clipped at 99th pct)")
plt.tight_layout()
plt.show()

# ===== Code cell 17 ========================================
corr_matrix = df[safe_cols].corr()
upper = corr_matrix.where(np.triu(np.ones_like(corr_matrix, dtype=bool), k=1))
pairs = upper.stack()
high = pairs[pairs.abs() > 0.8].sort_values(ascending=False)

print("feature pairs with |corr| > 0.8:")
print(high.round(3))

# ===== Code cell 18 ========================================
top12 = corrs.abs().sort_values(ascending=False).head(12).index

plt.figure(figsize=(9, 7))
sns.heatmap(
    df[list(top12)].corr(),
    annot=True,
    fmt=".2f",
    cmap="coolwarm",
    vmin=-1,
    vmax=1,
    center=0,
)
```

```
plt.title("Correlations among the 12 strongest safe features")
plt.tight_layout()
plt.show()

# ===== Code cell 19 ======================================
LEAKY_COLS = ["URLSimilarityIndex", "URL", "Domain", "Title"]

clean_features = [
    c for c in df.columns if c not in LEAKY_COLS + ["label", "TLD"]
]

manifest = {
    "target": "label",
    "dropped_leaky": LEAKY_COLS,
    "dropped_identifier": [],
    "categorical_raw": ["TLD"],
    "drop_duplicate_urls": True,
    "split_group_column": "Domain",
    "clean_numeric_features": clean_features,
}

out_path = Path("../data/01_feature_columns.json")
out_path.write_text(json.dumps(manifest, indent=2))
print(f"saved {len(clean_features)} feature names to {out_path}")
```

Notebook 02 – Classical Models (Jason Justice)

```
# Notebook 02 - Classical Models (Jason Justice)
# Exported from 02_classical_models.ipynb (code cells only, in order).

# ===== Code cell 1 =======================================
import json
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
from imblearn.over_sampling import SMOTE
from imblearn.pipeline import Pipeline as ImbPipeline
from rapidfuzz import process
from rapidfuzz.distance import Levenshtein
from sklearn.ensemble import RandomForestClassifier
from sklearn.inspection import permutation_importance
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score,
    average_precision_score,
    confusion_matrix,
    f1_score,
    precision_recall_curve,
    precision_score,
    recall_score,
    roc_auc_score,
```

```
)
from sklearn.model_selection import (
    GroupShuffleSplit,
    RandomizedSearchCV,
    StratifiedGroupKFold,
)
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from xgboost import XGBClassifier

RANDOM_STATE = 42

# ===== Code cell 2 ========================================
DATA_DIR = Path("../data")
df = pd.read_csv(DATA_DIR / "phiusiil.csv")
manifest = json.loads((DATA_DIR / "01_feature_columns.json").read_text())

clean_features = manifest["clean_numeric_features"]
print(f"rows: {len(df):,}")
print(f"clean features from notebook 01: {len(clean_features)}")

# ===== Code cell 3 ========================================
def shannon_entropy(s):
    if not s:
        return 0.0
    counts = np.unique(list(s), return_counts=True)[1]
    probs = counts / counts.sum()
    return float(-(probs * np.log2(probs)).sum())


SUFFIX_STOP = {
    "www",
    "com",
    "org",
    "net",
    "edu",
    "gov",
    "co",
    "uk",
    "io",
    "de",
    "ru",
    "info",
    "biz",
    "app",
    "xyz",
    "online",
    "site",
    "web",
}
SHORTENERS = {
    "bit.ly",
    "tinyurl.com",
    "goo.gl",
```

```
    "t.co",
    "ow.ly",
    "is.gd",
    "buff.ly",
    "adf.ly",
    "rebrand.ly",
    "cutt.ly",
    "shorturl.at",
}
BRANDS = [
    "paypal",
    "apple",
    "microsoft",
    "google",
    "amazon",
    "facebook",
    "instagram",
    "netflix",
    "linkedin",
    "whatsapp",
    "outlook",
    "office365",
    "chase",
    "wellsfargo",
    "bankofamerica",
    "citibank",
    "hsbc",
    "barclays",
    "americanexpress",
    "capitalone",
    "santander",
    "visa",
    "mastercard",
    "coinbase",
    "binance",
    "blockchain",
    "metamask",
    "dropbox",
    "adobe",
    "yahoo",
    "gmail",
    "icloud",
    "spotify",
    "steam",
    "roblox",
    "twitter",
    "tiktok",
    "snapchat",
    "ebay",
    "walmart",
    "target",
    "fedex",
    "ups",
    "dhl",
```

```
    "usps",
    "irs",
    "hmrc",
    "docusign",
    "onedrive",
    "zoom",
    "twitch",
    "discord",
    "telegram",
    "reddit",
    "github",
    "wordpress",
    "shopify",
    "stripe",
    "venmo",
    "zelle",
    "cashapp",
    "revolut",
    "aliexpress",
    "alibaba",
    "samsung",
    "xfinity",
    "verizon",
    "tmobile",
    "vodafone",
    "booking",
    "airbnb",
    "uber",
    "lyft",
    "doordash",
    "instacart",
]


def domain_labels(domain):
    return [
        p
        for p in domain.lower().split(".")
        if p not in SUFFIX_STOP and len(p) >= 3
    ]


def registered_domain(domain):
    d = domain.lower()
    return d[4:] if d.startswith("www.") else d

# ===== Code cell 4 ========================================
dom = df["Domain"].astype(str)

df["url_entropy"] = df["URL"].astype(str).map(shannon_entropy)
df["domain_entropy"] = dom.map(shannon_entropy)
df["is_punycode"] = dom.str.contains("xn--").astype(int)
df["is_shortener"] = dom.map(registered_domain).isin(SHORTENERS).astype(int)

```

```
# brand_distance: min normalized Levenshtein from any significant domain
# label to a brand. Compute once per unique label (fast in C), then map.
labels_per_row = dom.map(domain_labels)
unique_labels = sorted({lab for labs in labels_per_row for lab in labs})
dist = process.cdist(
    unique_labels, BRANDS, scorer=Levenshtein.normalized_distance
)
label_min = dict(zip(unique_labels, dist.min(axis=1)))


def brand_distance(labs):
    return min((label_min[x] for x in labs), default=1.0)


df["brand_distance"] = labels_per_row.map(brand_distance).astype(float)

engineered = [
    "url_entropy",
    "domain_entropy",
    "brand_distance",
    "is_punycode",
    "is_shortener",
]
print(df[engineered].describe().round(3).to_string())

# ===== Code cell 5 ========================================
phish = df["label"] == 0
by_class = pd.DataFrame(
    {
        "phishing_median": df.loc[phish, engineered].median(),
        "legit_median": df.loc[~phish, engineered].median(),
    }
)
print(by_class.round(3).to_string())

cont = ["url_entropy", "domain_entropy", "brand_distance"]
fig, axes = plt.subplots(1, 3, figsize=(13, 3.5))
for ax, col in zip(axes, cont):
    ax.hist(
        df.loc[~phish, col],
        bins=40,
        alpha=0.7,
        density=True,
        color="tab:blue",
        label="legitimate",
    )
    ax.hist(
        df.loc[phish, col],
        bins=40,
        alpha=0.7,
        density=True,
        color="tab:orange",
        label="phishing",
    )
```

```
    ax.set_title(col)
    ax.legend()
plt.tight_layout()
plt.show()

# ===== Code cell 6 ======================================
features = clean_features + engineered

before = len(df)
model_df = df.drop_duplicates(subset="URL").reset_index(drop=True)
print(f"dropped {before - len(model_df):,} duplicate-URL rows")

X = model_df[features]
y = 1 - model_df["label"]  # 1 = phishing (positive class)
groups = model_df["Domain"]

gss = GroupShuffleSplit(n_splits=1, test_size=0.2, random_state=RANDOM_STATE)
train_idx, test_idx = next(gss.split(X, y, groups=groups))
X_train, X_test = X.iloc[train_idx], X.iloc[test_idx]
y_train, y_test = y.iloc[train_idx], y.iloc[test_idx]
groups_train = groups.iloc[train_idx]

print(f"train: {len(X_train):,}   test: {len(X_test):,}")
print(f"phishing rate  train={y_train.mean():.3f}  test={y_test.mean():.3f}")

# ===== Code cell 7 ======================================
def xgb(**kw):
    params = dict(
        n_estimators=400,
        max_depth=6,
        learning_rate=0.1,
        tree_method="hist",
        eval_metric="logloss",
        n_jobs=-1,
        random_state=RANDOM_STATE,
    )
    params.update(kw)
    return XGBClassifier(**params)


def evaluate(name, model, X_te, y_te):
    proba = model.predict_proba(X_te)[:, 1]
    pred = (proba >= 0.5).astype(int)
    return {
        "model": name,
        "accuracy": accuracy_score(y_te, pred),
        "precision": precision_score(y_te, pred),
        "recall": recall_score(y_te, pred),
        "f1": f1_score(y_te, pred),
        "roc_auc": roc_auc_score(y_te, proba),
        "pr_auc": average_precision_score(y_te, proba),
    }


```

```
n_pos = int(y_train.sum())
n_neg = int((y_train == 0).sum())

models = {
    "LogReg": Pipeline(
        [
            ("scale", StandardScaler()),
            (
                "clf",
                LogisticRegression(max_iter=1000, class_weight="balanced"),
            ),
        ]
    ),
    "RandomForest": RandomForestClassifier(
        n_estimators=300,
        class_weight="balanced",
        n_jobs=-1,
        random_state=RANDOM_STATE,
    ),
    "XGBoost": xgb(scale_pos_weight=n_neg / n_pos),
}

rows = []
for name, model in models.items():
    model.fit(X_train, y_train)
    rows.append(evaluate(name, model, X_test, y_test))

results = pd.DataFrame(rows).set_index("model")
print(results.round(4).to_string())

# ===== Code cell 8 ========================================
fig, axes = plt.subplots(1, 3, figsize=(13, 3.6))
for ax, (name, model) in zip(axes, models.items()):
    cm = confusion_matrix(y_test, model.predict(X_test))
    sns.heatmap(
        cm,
        annot=True,
        fmt=",d",
        cmap="Blues",
        cbar=False,
        ax=ax,
        xticklabels=["legit", "phish"],
        yticklabels=["legit", "phish"],
    )
    ax.set_title(name)
    ax.set_xlabel("predicted")
    ax.set_ylabel("actual")
plt.tight_layout()
plt.show()

# ===== Code cell 9 ========================================
ablation = {
    "none": xgb(),
    "scale_pos_weight": xgb(scale_pos_weight=n_neg / n_pos),
```

```
    "SMOTE": ImbPipeline(
        [
            ("smote", SMOTE(random_state=RANDOM_STATE)),
            ("clf", xgb()),
        ]
    ),
}

rows = []
for name, model in ablation.items():
    model.fit(X_train, y_train)
    rows.append(evaluate(name, model, X_test, y_test))

print(pd.DataFrame(rows).set_index("model").round(4).to_string())

# ===== Code cell 10 ========================================
cv = StratifiedGroupKFold(n_splits=5)
param_dist = {
    "n_estimators": [200, 400, 600],
    "max_depth": [4, 6, 8, 10],
    "learning_rate": [0.03, 0.1, 0.3],
    "subsample": [0.7, 0.9, 1.0],
    "colsample_bytree": [0.7, 0.9, 1.0],
}
search = RandomizedSearchCV(
    xgb(scale_pos_weight=n_neg / n_pos),
    param_distributions=param_dist,
    n_iter=10,
    scoring="average_precision",
    cv=cv,
    random_state=RANDOM_STATE,
    n_jobs=-1,
)
search.fit(X_train, y_train, groups=groups_train)
best = search.best_estimator_

print("best params:", search.best_params_)
print(f"CV PR-AUC: {search.best_score_:.4f}")
tuned = evaluate("XGBoost (tuned)", best, X_test, y_test)
print(pd.DataFrame([tuned]).set_index("model").round(4).to_string())

# ===== Code cell 11 ========================================
importances = pd.Series(best.feature_importances_, index=features).sort_values(
    ascending=False
)

plt.figure(figsize=(8, 5))
importances.head(15)[::-1].plot.barh(color="tab:blue")
plt.xlabel("XGBoost gain importance")
plt.title("Top 15 features (tuned XGBoost)")
plt.tight_layout()
plt.show()

print(f"engineered-feature ranks (of {len(features)} features):")
```

```
for f in engineered:
    rank = int(importances.index.get_loc(f)) + 1
    print(f"  {f:16s} rank {rank:2d}   gain {importances[f]:.4f}")

# ===== Code cell 12 ======================================
sample = X_test.sample(min(20000, len(X_test)), random_state=RANDOM_STATE)
perm = permutation_importance(
    best,
    sample,
    y_test.loc[sample.index],
    scoring="average_precision",
    n_repeats=5,
    random_state=RANDOM_STATE,
    n_jobs=-1,
)
perm_imp = pd.Series(perm.importances_mean, index=features)
print("top 10 by permutation importance (PR-AUC drop):")
print(perm_imp.sort_values(ascending=False).head(10).round(4).to_string())

# ===== Code cell 13 ======================================
K = 8
top_k = importances.head(K).index.tolist()
print(f"top {K} features: {top_k}")

simple = xgb(scale_pos_weight=n_neg / n_pos)
simple.fit(X_train[top_k], y_train)
simple_res = evaluate(f"XGBoost (top {K})", simple, X_test[top_k], y_test)

compare = pd.DataFrame([tuned, simple_res]).set_index("model")
print(compare.round(4).to_string())

# ===== Code cell 14 ======================================
# Illustrative per-event costs (see markdown above).
COST_FN = 500.0
COST_FP = 5.0

proba = best.predict_proba(X_test)[:, 1]
thresholds = np.linspace(0.01, 0.99, 99)
n = len(y_test)


def cost_per_1k(t):
    pred = (proba >= t).astype(int)
    fn = int(((pred == 0) & (y_test == 1)).sum())
    fp = int(((pred == 1) & (y_test == 0)).sum())
    return (COST_FN * fn + COST_FP * fp) / n * 1000


cost_curve = np.array([cost_per_1k(t) for t in thresholds])
best_t = float(thresholds[cost_curve.argmin()])

plt.figure(figsize=(8, 4))
plt.plot(thresholds, cost_curve, color="tab:blue")
plt.axvline(best_t, color="tab:red", ls="--", label=f"min-cost t={best_t:.2f}")
```

```
plt.axvline(0.5, color="gray", ls=":", label="default t=0.50")
plt.xlabel("threshold (flag phishing if proba >= t)")
plt.ylabel("expected cost per 1,000 URLs ($)")
plt.title("Cost-sensitive threshold selection")
plt.legend()
plt.tight_layout()
plt.show()

print(
    f"min-cost threshold: {best_t:.2f}  "
    f"(${cost_curve.min():.2f} per 1k) vs "
    f"${cost_per_1k(0.5):.2f} at default 0.5"
)

# ===== Code cell 15 ========================================
prec, rec, thr = precision_recall_curve(y_test, proba)


def operating_point(target, mode):
    metric = prec[:-1] if mode == "precision" else rec[:-1]
    other = rec[:-1] if mode == "precision" else prec[:-1]
    ok = metric >= target
    if not ok.any():
        return None
    idx = np.where(ok)[0]
    pick = idx[np.argmax(other[idx])]
    return thr[pick], prec[pick], rec[pick]


for name, mode in [
    ("precision-first (prec>=0.99)", "precision"),
    ("recall-first (recall>=0.99)", "recall"),
]:
    r = operating_point(0.99, mode)
    if r:
        t, p, rc = r
        print(f"{name:30s} t={t:.3f}  precision={p:.4f}  recall={rc:.4f}")

# 3-tier policy
t_block, t_warn = 0.90, 0.50
tier = np.where(
    proba >= t_block,
    "block",
    np.where(proba >= t_warn, "warn", "allow"),
)
print()
print("3-tier policy on the test set:")
print(pd.Series(tier).value_counts().to_string())

# ===== Code cell 16 ========================================
all_results = pd.concat(
    [results, pd.DataFrame([tuned, simple_res]).set_index("model")]
)
out = {
```

```
    "notebook": "02",
    "positive_class": "phishing",
    "n_train": int(len(X_train)),
    "n_test": int(len(X_test)),
    "engineered_features": engineered,
    "metrics": all_results.round(4).reset_index().to_dict(orient="records"),
    "min_cost_threshold": best_t,
}
(DATA_DIR / "02_classical_results.json").write_text(json.dumps(out, indent=2))
print("saved data/02_classical_results.json")
```

Notebook 03 – CharCNN, Clustering, SOTA (Pranav Dhinakar)

```
# Notebook 03 - CharCNN, Clustering, SOTA (Pranav Dhinakar)
# Exported from 03_charcnn_clustering_sota.ipynb (code cells only, in order).

# ===== Code cell 1 =======================================
import json
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

# Section-specific heavy deps (torch, tldextract, hdbscan, rapidfuzz)
# are imported at the top of their own sections to keep setup light.

DATA_PATH = Path("../data/phiusiil.csv")
MANIFEST_PATH = Path("../data/01_feature_columns.json")

# ===== Code cell 2 =======================================
df = pd.read_csv(DATA_PATH)
manifest = json.loads(MANIFEST_PATH.read_text())

target = manifest["target"]
clean_features = manifest["clean_numeric_features"]
group_col = manifest["split_group_column"]

# Same pre-split cleaning NB 02 uses, so our numbers are comparable.
# We keep URL and Domain as columns (raw material for the char model,
# engineered features and grouping) even though they're not features.
if manifest["drop_duplicate_urls"]:
    before = len(df)
    df = df.drop_duplicates(subset="URL").reset_index(drop=True)
    print(f"dropped {before - len(df)} duplicate-URL rows")

y = df[target]

print(f"rows: {len(df):,}")
print(f"clean feature count: {len(clean_features)}")
print(f"group column for splits: {group_col}")

# ===== Code cell 3 =======================================
import tldextract
```

```python

# eTLD+1, e.g. "gateway.ipfs.io" -> "ipfs.io". Uses a bundled public
# suffix list; suppress the network refresh for reproducibility.
extractor = tldextract.TLDExtract(suffix_list_urls=())


def registered_domain(host: str) -> str:
    ext = extractor(str(host))
    # fall back to the raw host for IP-address URLs (no eTLD+1)
    return ext.top_domain_under_public_suffix or str(host)


df["RegDomain"] = df["Domain"].map(registered_domain)

n_exact = df["Domain"].nunique()
n_reg = df["RegDomain"].nunique()
print(f"exact Domain groups:     {n_exact:,}")
print(f"registered-domain groups: {n_reg:,}")
print(f"collapsed by {n_exact - n_reg:,} groups")
print()
# how many rows share a registered domain with another row?
reg_counts = df["RegDomain"].value_counts()
reg_repeated = reg_counts[reg_counts > 1]
print(
    f"rows sharing a registered domain: "
    f"{reg_repeated.sum():,} ({reg_repeated.sum() / len(df):.1%})"
)
reg_counts.head(10)

# ===== Code cell 4 ========================================
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, f1_score
from sklearn.model_selection import GroupShuffleSplit
from sklearn.preprocessing import StandardScaler


def quick_logreg(X, y, groups):
    """Basic logistic regression under a grouped split; return acc, F1."""
    gss = GroupShuffleSplit(n_splits=1, test_size=0.2, random_state=42)
    train_idx, test_idx = next(gss.split(X, y, groups=groups))
    X_train, X_test = X.iloc[train_idx], X.iloc[test_idx]
    y_train, y_test = y.iloc[train_idx], y.iloc[test_idx]

    scaler = StandardScaler()
    X_train = scaler.fit_transform(X_train)
    X_test = scaler.transform(X_test)
    model = LogisticRegression(max_iter=1000, class_weight="balanced")
    model.fit(X_train, y_train)
    preds = model.predict(X_test)
    return accuracy_score(y_test, preds), f1_score(y_test, preds)


X = df[clean_features]
acc_exact, f1_exact = quick_logreg(X, y, groups=df["Domain"])
```

```
acc_reg, f1_reg = quick_logreg(X, y, groups=df["RegDomain"])

print(f"exact-Domain split:      acc={acc_exact:.4f}  f1={f1_exact:.4f}")
print(f"registered-domain split: acc={acc_reg:.4f}  f1={f1_reg:.4f}")

# ===== Code cell 5 ======================================
# optional: treat multi-tenant PaaS suffixes as public, so
# victim-a.web.app and victim-b.web.app become separate groups
extractor_strict = tldextract.TLDExtract(
    suffix_list_urls=(), include_psl_private_domains=True
)
df["RegDomainStrict"] = df["Domain"].map(
    lambda h: extractor_strict(str(h)).top_domain_under_public_suffix or str(h)
)
print(f"strict (PaaS-aware) groups: {df['RegDomainStrict'].nunique():,}")

# ===== Code cell 6 ======================================
acc_strict, f1_strict = quick_logreg(X, y, groups=df["RegDomainStrict"])
print(f"PaaS-aware split: acc={acc_strict:.4f}  f1={f1_strict:.4f}")

# ===== Code cell 7 ======================================
# Build a character vocabulary from the training URLs only (fit on
# train to avoid leaking test-set characters into the vocab). Reserve
# 0 for padding and 1 for unknown/out-of-vocab characters.
MAX_LEN = 200  # covers ~99th pct of URL lengths; longer URLs truncated

urls = df["URL"].astype(str).values
labels = df[target].values

lengths = np.array([len(u) for u in urls])
print(
    f"URL length: median={np.median(lengths):.0f}  "
    f"95th={np.percentile(lengths, 95):.0f}  "
    f"99th={np.percentile(lengths, 99):.0f}  max={lengths.max()}"
)
print(
    f"URLs longer than MAX_LEN={MAX_LEN}: "
    f"{(lengths > MAX_LEN).sum()} ({(lengths > MAX_LEN).mean():.2%})"
)

# ===== Code cell 8 ======================================
from collections import Counter


# Phishing = positive, matching NB 02. df is already dup-URL-dropped and
# index-reset (Section 1), so this split reproduces NB 02's test set.
y_phish = (1 - df[target]).to_numpy()
groups = df["Domain"].to_numpy()
urls = df["URL"].astype(str).to_numpy()

gss = GroupShuffleSplit(n_splits=1, test_size=0.2, random_state=42)
train_idx, test_idx = next(gss.split(df, y_phish, groups=groups))

url_train, url_test = urls[train_idx], urls[test_idx]
```

```
y_train, y_test = y_phish[train_idx], y_phish[test_idx]

print(f"train: {len(train_idx):,}   test: {len(test_idx):,}")
print(f"phishing rate  train={y_train.mean():.3f}  test={y_test.mean():.3f}")

# Character vocabulary, fit on TRAIN urls only. 0 = <pad>, 1 = <unk>.
char_counts = Counter()
for u in url_train:
    char_counts.update(u)

MIN_FREQ = 5  # chars rarer than this in train collapse to <unk>
vocab = {"<pad>": 0, "<unk>": 1}
for ch, cnt in char_counts.most_common():
    if cnt >= MIN_FREQ:
        vocab[ch] = len(vocab)

print(f"distinct chars in train: {len(char_counts)}")
print(f"vocab size (>= {MIN_FREQ} occ, + pad/unk): {len(vocab)}")

# ===== Code cell 9 ========================================
def encode(url, vocab, max_len=MAX_LEN):
    ids = [vocab.get(c, 1) for c in url[:max_len]]  # 1 = <unk>
    ids += [0] * (max_len - len(ids))  # 0 = <pad>
    return ids


X_train_ids = np.array([encode(u, vocab) for u in url_train], dtype=np.int64)
X_test_ids = np.array([encode(u, vocab) for u in url_test], dtype=np.int64)

trunc = (np.array([len(u) for u in url_train]) > MAX_LEN).mean()
print(f"X_train_ids: {X_train_ids.shape}   X_test_ids: {X_test_ids.shape}")
print(f"train URLs truncated at {MAX_LEN}: {trunc:.2%}")
print("example row (first 40 ids):", X_train_ids[0][:40])

# ===== Code cell 10 =======================================
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import DataLoader, TensorDataset
from sklearn.model_selection import GroupShuffleSplit

torch.manual_seed(42)
np.random.seed(42)

if torch.cuda.is_available():
    device = "cuda"
elif torch.backends.mps.is_available():
    device = "mps"
else:
    device = "cpu"
print(f"device: {device}")

# Inner grouped train/val split; val only monitors convergence / early
# stopping. Grouped so no domain straddles train and val either.
```

```
groups_train = groups[train_idx]
inner = GroupShuffleSplit(n_splits=1, test_size=0.1, random_state=42)
tr_i, va_i = next(inner.split(X_train_ids, y_train, groups=groups_train))

Xtr = torch.tensor(X_train_ids[tr_i], dtype=torch.long)
ytr = torch.tensor(y_train[tr_i], dtype=torch.float32)
Xva = torch.tensor(X_train_ids[va_i], dtype=torch.long)
yva = torch.tensor(y_train[va_i], dtype=torch.float32)
Xte = torch.tensor(X_test_ids, dtype=torch.long)
yte = torch.tensor(y_test, dtype=torch.float32)

BATCH = 512
train_loader = DataLoader(
    TensorDataset(Xtr, ytr), batch_size=BATCH, shuffle=True
)
val_loader = DataLoader(TensorDataset(Xva, yva), batch_size=BATCH)
test_loader = DataLoader(TensorDataset(Xte, yte), batch_size=BATCH)

# class weight for phishing (positive): n_neg / n_pos on the train split
n_pos = float(ytr.sum())
n_neg = float(len(ytr) - ytr.sum())
pos_weight = torch.tensor([n_neg / n_pos], device=device)
print(
    f"train/val: {len(tr_i):,}/{len(va_i):,}   pos_weight={n_neg / n_pos:.3f}"
)


class CharCNN(nn.Module):
    def __init__(
        self,
        vocab_size,
        embed_dim=48,
        n_filters=128,
        kernels=(3, 5, 7),
        dropout=0.4,
    ):
        super().__init__()
        self.embed = nn.Embedding(vocab_size, embed_dim, padding_idx=0)
        self.convs = nn.ModuleList(
            [
                nn.Conv1d(embed_dim, n_filters, k, padding=k // 2)
                for k in kernels
            ]
        )
        self.dropout = nn.Dropout(dropout)
        self.fc = nn.Linear(n_filters * len(kernels), 1)

    def forward(self, x):
        e = self.embed(x).permute(0, 2, 1)  # (B, embed, L)
        feats = [
            F.relu(conv(e)).max(dim=2).values  # global max-pool
            for conv in self.convs
        ]
        h = self.dropout(torch.cat(feats, dim=1))
```

```
        return self.fc(h).squeeze(1)  # (B,) logits


model = CharCNN(vocab_size=len(vocab)).to(device)
n_params = sum(p.numel() for p in model.parameters())
print(f"model params: {n_params:,}")

# ===== Code cell 11 ======================================
from sklearn.metrics import average_precision_score

criterion = nn.BCEWithLogitsLoss(pos_weight=pos_weight)
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
EPOCHS = 6


@torch.no_grad()
def predict_proba(loader):
    model.eval()
    probs = []
    for xb, _ in loader:
        logits = model(xb.to(device))
        probs.append(torch.sigmoid(logits).cpu().numpy())
    return np.concatenate(probs)


best_val_ap, best_state = -1.0, None
for epoch in range(1, EPOCHS + 1):
    model.train()
    total = 0.0
    for xb, yb in train_loader:
        xb, yb = xb.to(device), yb.to(device)
        optimizer.zero_grad()
        loss = criterion(model(xb), yb)
        loss.backward()
        optimizer.step()
        total += loss.item() * len(xb)
    train_loss = total / len(Xtr)

    val_p = predict_proba(val_loader)
    val_ap = average_precision_score(yva.numpy(), val_p)
    val_f1 = f1_score(yva.numpy(), (val_p >= 0.5).astype(int))
    flag = ""
    if val_ap > best_val_ap:
        best_val_ap = val_ap
        best_state = {
            k: v.detach().cpu().clone() for k, v in model.state_dict().items()
        }
        flag = "  <- best"
    print(
        f"epoch {epoch}  train_loss={train_loss:.4f}  "
        f"val_PR_AUC={val_ap:.4f}  val_F1={val_f1:.4f}{flag}"
    )

model.load_state_dict(best_state)
```

```
print(f"\nrestored best val PR-AUC = {best_val_ap:.4f}")

# ===== Code cell 12 ========================================
import seaborn as sns
from sklearn.metrics import (
    confusion_matrix,
    precision_score,
    recall_score,
    roc_auc_score,
)

test_proba = predict_proba(test_loader)
test_pred = (test_proba >= 0.5).astype(int)

char_result = {
    "model": "CharCNN (URL only)",
    "accuracy": accuracy_score(y_test, test_pred),
    "precision": precision_score(y_test, test_pred),
    "recall": recall_score(y_test, test_pred),
    "f1": f1_score(y_test, test_pred),
    "roc_auc": roc_auc_score(y_test, test_proba),
    "pr_auc": average_precision_score(y_test, test_proba),
}
print(pd.Series(char_result).to_string())

cm = confusion_matrix(y_test, test_pred)
plt.figure(figsize=(4, 3.4))
sns.heatmap(
    cm,
    annot=True,
    fmt=",d",
    cmap="Blues",
    cbar=False,
    xticklabels=["legit", "phish"],
    yticklabels=["legit", "phish"],
)
plt.title("CharCNN (URL only): test")
plt.xlabel("predicted")
plt.ylabel("actual")
plt.tight_layout()
plt.show()

# ===== Code cell 13 ========================================
# Cluster the PHISHING rows only, on the leakage-clean feature set from
# NB 01 (label held out). Goal: find structure within phishing,
# "attack families", not to separate phishing from legit.
phish_mask = (df[target] == 0).to_numpy()
Xp_raw = df.loc[phish_mask, clean_features].copy()

# Drop zero-variance columns within the phishing subset (constant here,
# so they carry no clustering signal and would clutter the profile).
nunique = Xp_raw.nunique()
const_cols = nunique[nunique <= 1].index.tolist()
Xp_raw = Xp_raw.drop(columns=const_cols)
```

```
366 cluster_features = Xp_raw.columns.tolist()
367
368 scaler = StandardScaler()
369 Xp = scaler.fit_transform(Xp_raw)
370
371 print(f"phishing rows: {Xp.shape[0]:,}")
372 print(
373     f"clustering features: {Xp.shape[1]} "
374     f"(dropped {len(const_cols)} constant: {const_cols})"
375 )
376
377 # ===== Code cell 14 ========================================
378 from sklearn.cluster import KMeans
379 from sklearn.metrics import silhouette_score
380
381 ks = range(2, 11)
382 inertias, silhouettes = [], []
383 for k in ks:
384     km = KMeans(n_clusters=k, n_init=10, random_state=42)
385     lbl = km.fit_predict(Xp)
386     inertias.append(km.inertia_)
387     # silhouette on a subsample; full O(n^2) is infeasible at ~100k rows
388     sil = silhouette_score(Xp, lbl, sample_size=10000, random_state=42)
389     silhouettes.append(sil)
390     print(f"k={k}  inertia={km.inertia_:,.0f}  silhouette={sil:.4f}")
391
392 fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4))
393 ax1.plot(list(ks), inertias, "o-", color="tab:blue")
394 ax1.set_xlabel("k")
395 ax1.set_ylabel("inertia (within-cluster SS)")
396 ax1.set_title("Elbow")
397 ax2.plot(list(ks), silhouettes, "o-", color="tab:orange")
398 ax2.set_xlabel("k")
399 ax2.set_ylabel("silhouette (10k sample)")
400 ax2.set_title("Silhouette")
401 plt.tight_layout()
402 plt.show()
403
404 # ===== Code cell 15 ========================================
405 K_FAMILIES = 5
406
407 # Dominant structure is binary (k=2, silhouette 0.44); characterize it
408 # briefly, then use k=5 for interpretable families.
409 km2 = KMeans(n_clusters=2, n_init=10, random_state=42).fit(Xp)
410 sizes2 = pd.Series(km2.labels_).value_counts().sort_index()
411 print("k=2 dominant split sizes:")
412 for c, n in sizes2.items():
413     print(f"  cluster {c}: {n:,} ({n / len(Xp):.1%})")
414
415 km5 = KMeans(n_clusters=K_FAMILIES, n_init=10, random_state=42)
416 phish_labels = km5.fit_predict(Xp)
417
418 profile = Xp_raw.copy()
419 profile["cluster"] = phish_labels
```

```
sizes = pd.Series(phish_labels).value_counts().sort_index()
print(f"\nk={K_FAMILIES} family sizes:")
for c, n in sizes.items():
    print(f"  cluster {c}: {n:,} ({n / len(Xp):.1%})")

# Interpretable raw-unit profile (only columns present in the feature set)
shortlist = [
    "IsDomainIP",
    "IsHTTPS",
    "URLLength",
    "DomainLength",
    "NoOfSubDomain",
    "DegitRatioInURL",
    "SpacialCharRatioInURL",
    "LineOfCode",
    "NoOfJS",
    "NoOfImage",
    "NoOfExternalRef",
    "NoOfCSS",
    "HasSocialNet",
    "HasCopyrightInfo",
    "HasDescription",
    "HasPasswordField",
    "HasSubmitButton",
    "TLDLegitimateProb",
]
shortlist = [c for c in shortlist if c in cluster_features]
means = profile.groupby("cluster")[shortlist].mean().T
means.columns = [f"c{c}" for c in means.columns]
print(f"\nper-cluster mean (raw units), {len(shortlist)} features:")
print(means.round(2).to_string())

# ===== Code cell 16 ========================================
# KMeans centroids live in standardized space, so each value is that
# cluster's mean in SDs from the phishing-wide mean. Large |z| = what
# makes the family distinctive.
centroids = pd.DataFrame(km5.cluster_centers_, columns=cluster_features)
for c in range(K_FAMILIES):
    row = centroids.loc[c]
    top = row.reindex(row.abs().sort_values(ascending=False).index).head(6)
    print(f"cluster {c} (n={sizes[c]:,}), top distinguishing features:")
    for feat, z in top.items():
        arrow = "↑" if z > 0 else "↓"
        print(f"    {feat:24s} {z:+.2f} {arrow}")
    print()

# ===== Code cell 17 ========================================
from sklearn.decomposition import PCA

FAMILY_NAMES = {
    0: "empty-shell (68%)",
    1: "outlier long-URL (5)",
    2: "IP-host / obfuscated (1%)",
    3: "credential forms (12%)",
```

```
    4: "HTTP lookalike stubs (19%)",
}

pca = PCA(n_components=2, random_state=42)
coords = pca.fit_transform(Xp)
ev = pca.explained_variance_ratio_
print(
    f"PC1+PC2 explained variance: {ev[0]:.1%} + {ev[1]:.1%} = {ev.sum():.1%}"
)

# Subsample for a readable scatter; fit was on all rows.
rng = np.random.default_rng(42)
idx = rng.choice(len(coords), size=min(15000, len(coords)), replace=False)

plt.figure(figsize=(8, 6))
for c in range(K_FAMILIES):
    m = phish_labels[idx] == c
    plt.scatter(
        coords[idx][m, 0],
        coords[idx][m, 1],
        s=4,
        alpha=0.4,
        label=FAMILY_NAMES[c],
    )
plt.xlabel(f"PC1 ({ev[0]:.0%} var)")
plt.ylabel(f"PC2 ({ev[1]:.0%} var)")
plt.title("Phishing families: PCA projection (15k sample)")
plt.legend(markerscale=3, fontsize=8, loc="best")
plt.tight_layout()
plt.show()

# ===== Code cell 18 ========================================
from sklearn.cluster import HDBSCAN

# Density-based cross-check on a subsample (HDBSCAN is heavier than
# KMeans). Unlike KMeans it doesn't force every point into a cluster;
# it can label sparse outliers as noise (-1), which is the honest test
# of how much crisp structure exists and whether the 5-row KMeans
# micro-cluster survives.
sub = rng.choice(len(Xp), size=min(25000, len(Xp)), replace=False)
hdb = HDBSCAN(min_cluster_size=250, min_samples=10)
hdb_labels = hdb.fit_predict(Xp[sub])

n_clusters = len(set(hdb_labels)) - (1 if -1 in hdb_labels else 0)
noise = (hdb_labels == -1).mean()
print(f"HDBSCAN on {len(sub):,}-row sample")
print(f"clusters found: {n_clusters}   noise: {noise:.1%}")
print()
sizes_hdb = pd.Series(hdb_labels).value_counts().sort_index()
for c, n in sizes_hdb.items():
    tag = " (noise)" if c == -1 else ""
    print(f"  label {c:>2}: {n:,} ({n / len(sub):.1%}){tag}")

# ===== Code cell 19 ========================================
```

```
# NB 02's saved tabular metrics (phishing = positive, same exact-Domain
# split as this notebook, so the rows below are directly comparable).
nb02_path = Path("../data/02_classical_results.json")
assert nb02_path.exists(), (
    "Run NB 02 first; it saves 02_classical_results.json this table needs."
)
nb02 = json.loads(nb02_path.read_text())
assert nb02["positive_class"] == "phishing"

tab = pd.DataFrame(nb02["metrics"]).set_index("model")

# Append this notebook's CharCNN result (live variable from above).
char_row = (
    pd.Series(char_result).drop("model").to_frame(char_result["model"]).T
)
metric_cols = ["accuracy", "precision", "recall", "f1", "roc_auc", "pr_auc"]
our = pd.concat([tab, char_row])[metric_cols].astype(float)

print(
    f"NB 02 test rows: {nb02['n_test']:,}   "
    f"CharCNN test rows: {len(y_test):,}  (shared split)\n"
)
print(our.round(4).to_string())

# Graphical empirical comparison (rubric asks for this explicitly).
# Zoom the y-axis since every model is near the ceiling.
plot_metrics = ["accuracy", "recall", "pr_auc"]
lo = float(our[plot_metrics].min().min())

ax = our[plot_metrics].plot.bar(
    figsize=(11, 4.5),
    width=0.82,
    color=["tab:blue", "tab:orange", "tab:green"],
)
ax.set_ylim(max(0.0, lo - 0.01), 1.001)
ax.set_ylabel("score")
ax.set_title("Our models: key metrics (test set, phishing = positive)")
ax.legend(loc="lower right")
plt.xticks(rotation=25, ha="right")
plt.tight_layout()
plt.show()

# ===== Code cell 20 ======================================
nb03_out = {
    "notebook": "03",
    "positive_class": "phishing",
    "split": {
        "grouping": "exact Domain (matches NB 02)",
        "test_size": 0.2,
        "random_state": 42,
        "n_test": int(len(y_test)),
        "grouping_invariance": {
            "exact_domain": 0.9995,
            "paas_aware_etld1": 0.9993,
```

```
            "naive_etld1": 0.9990,
        },
    },
    "charcnn": {
        "description": "char-level CNN on raw URL string only, no "
        "engineered/content/similarity features",
        "max_len": int(MAX_LEN),
        "vocab_size": int(len(vocab)),
        "n_params": int(n_params),
        "metrics": {
            k: float(v) for k, v in char_result.items() if k != "model"
        },
        "confusion_matrix": confusion_matrix(y_test, test_pred).tolist(),
    },
    "clustering": {
        "algorithm": "KMeans",
        "k": int(K_FAMILIES),
        "n_phishing_rows": int(Xp.shape[0]),
        "silhouette_k2": 0.4373,
        "silhouette_k5": 0.1540,
        "families": {
            FAMILY_NAMES[c]: int(sizes[c]) for c in range(K_FAMILIES)
        },
        "hdbscan_check": {
            "sample_size": int(len(sub)),
            "clusters": int(n_clusters),
            "noise_fraction": float(noise),
        },
    },
    "our_models_vs_literature": {
        "our_models": our.round(4)
        .reset_index()
        .rename(columns={"index": "model"})
        .to_dict(orient="records"),
        "phiusiil_published_band": "99.5-100%",
        "url_only_literature_band": "97-99% (other corpora)",
    },
}

out_path = Path("../data/03_charcnn_cluster_results.json")
out_path.write_text(json.dumps(nb03_out, indent=2))
print(f"saved {out_path}")
print(
    f"  charcnn: {nb03_out['charcnn']['metrics']['accuracy']:.4f} acc, "
    f"{nb03_out['charcnn']['metrics']['pr_auc']:.4f} PR-AUC"
)
print(f"  families: {nb03_out['clustering']['families']}")
```